

Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Instytut Mikroelektroniki i Optoelektroniki

Praca dyplomowa inżynierska

na kierunku Elektronika i Telekomunikacja
w specjalności Inżynieria Komputerowa

Zastosowanie wybranych algorytmów uczenia maszynowego
w zarządzaniu łańcuchem dostaw

Jarosław Wanczewski

Numer albumu 309470

promotor
dr inż. Marek Niewiński

WARSZAWA 2026

Zastosowanie wybranych algorytmów uczenia maszynowego w zarządzaniu łańcuchem dostaw

Streszczenie

Praca dotyczy zastosowania algorytmów uczenia maszynowego w zarządzaniu łańcuchem dostaw, koncentrując się na rozwiązaniu problemów związanych z prognozowaniem popytu oraz zarządzaniem zapasami. W pierwszej części przedstawione zostaje pojęcie zarządzania łańcuchem dostaw, a następnie opisane są wybrane jego elementy. Przybliżone zostają również problemy napotykane w prognozowaniu popytu i zarządzaniu zapasami przy stosowaniu tradycyjnych metod.

W kolejnym rozdziale omówione zostaje pojęcie uczenia maszynowego, a następnie przedstawione jest jego zastosowanie w zarządzaniu łańcuchem dostaw. Opisane zostaje działanie kluczowych algorytmów uczenia maszynowego (nadzorowanego i nienadzorowanego). Na końcu rozdziału dokonany zostaje wybór algorytmów do rozwiązania problemów wybranych elementów łańcucha dostaw.

W dalszej części pracy przedstawione zostaje przygotowane narzędzie analityczne umożliwiające przeprowadzanie eksperymentów prognozowania popytu oraz symulacji kontroli zapasów. Opisany zostaje proces przygotowania danych w środowisku testowym.

Następnie zaprezentowane zostaje studium przypadku, w którym wykorzystano wybrane algorytmy uczenia maszynowego do prognozowania popytu oraz analizy wpływu jakości prognoz na zarządzanie zapasami. Przedstawione zostają dane wykorzystane w testach oraz wyniki eksperymentów zarówno dla tradycyjnych metod prognozowania, jak i modeli uczenia maszynowego.

W ostatnim rozdziale przedstawione zostają wyniki oraz sformułowane zostają wnioski wynikające z przeprowadzonych analiz.

Słowa kluczowe: uczenie maszynowe, łańcuch dostaw, prognozowanie popytu, zarządzanie zapasami

Application of Selected Machine Learning Algorithms in Supply Chain Management

Abstract

The thesis concerns the application of machine learning algorithms in supply chain management, focusing on solving problems related to demand forecasting and inventory management. In the first part, the concept of supply chain management is introduced, followed by a description of its selected components. The problems encountered in demand forecasting and inventory management when using traditional methods are also outlined.

In the next chapter, the concept of machine learning is discussed, followed by an explanation of its application in supply chain management. The operation of key machine learning algorithms (both supervised and unsupervised) is described. At the end of the chapter, a selection of algorithms suitable for addressing problems in selected elements of the supply chain is presented.

In the subsequent part of the thesis, a research tool developed for conducting demand forecasting experiments and inventory control simulations is presented. The process of data preparation within the test environment is also described.

Next, a case study is presented in which selected machine learning algorithms are applied to demand forecasting and to analyze the impact of forecast accuracy on inventory management. The data used in the experiments as well as the results obtained for both traditional forecasting methods and machine learning models are presented.

In the final chapter, the results are presented and conclusions are formulated based on the conducted analyses.

Keywords: machine learning, supply chain, demand forecasting, inventory management

Spis treści

1. Wprowadzenie	8
2. Cel i zakres pracy	9
3. Zarządzanie łańcuchem dostaw	10
4. Uczenie maszynowe	14
4.1. Metody uczenia maszynowego i ich zastosowania	14
4.2. Wybrane algorytmy uczenia maszynowego	18
5. Dobór narzędzi i przygotowanie środowiska analitycznego	23
6. Testy modeli prognozowania popytu i zarządzania zapasami	29
6.1. Test prognozowania popytu	29
6.2. Test zarządzania zapasami	41
7. Wyniki i podsumowanie	44
Bibliografia	47
Wykaz symboli i skrótów	51
Spis rysunków	55
Spis tabel	56
Spis załączników	56

1. Wprowadzenie

Zarządzanie łańcuchem dostaw stało się jednym z najważniejszych źródeł przewagi konkurencyjnej współczesnych przedsiębiorstw. Zdolność do szybkiego i precyzyjnego reagowania na zmiany popytu bezpośrednio przekłada się na poziom kosztów, rotację zapasów oraz satysfakcję klientów.

Jednym z najtrudniejszych i jednocześnie najbardziej kosztotwórczych elementów tego zarządzania jest prognozowanie popytu oraz wynikające z niego decyzje dotyczące poziomu zapasów. Zbyt pesymistyczna prognoza prowadzi do braków i utraty przychodów, zbyt optymistyczna – do nadmiernych zapasów, zamrożonego kapitału i wysokich kosztów magazynowania.

Tradycyjne metody prognozowania opierają się często na uproszczonych modelach statystycznych, które wykorzystują prawie wyłącznie dane historyczne sprzedaży i nie uwzględniają dodatkowych zmiennych opisujących sytuację (promocje, kalendarz, święta, pogoda, akcje marketingowe, ceny konkurentów itp.). W rezultacie ich dokładność w rzeczywistych warunkach jest często niewystarczająca. Algorytmy uczenia maszynowego umożliwiają tworzenie modeli prognozujących popyt na podstawie danych historycznych oraz cech dodatkowych, takich jak opóźnienia, cechy kalendarzowe i statystyki kroczące. Uzyskane prognozy mogą być następnie wykorzystane jako dane wejściowe do symulacji kontroli zapasów, w której analizowany jest wpływ dokładności prognozy na liczbę dni braku zapasu oraz średni poziom zapasu.

2. Cel i zakres pracy

Celem pracy jest ocena możliwości zastosowania wybranych algorytmów uczenia maszynowego w zarządzaniu łańcuchem dostaw w dwóch obszarach: prognozowaniu popytu oraz zarządzaniu zapasami. Cel ten obejmuje porównanie jakości prognoz uzyskanych metodami uczenia maszynowego z wynikami metod tradycyjnych oraz analizę, w jakim stopniu jakość prognozy wpływa na wskaźniki magazynowe, takie jak liczba dni braku zapasu oraz średni poziom zapasu.

Zakres pracy obejmuje przygotowanie środowiska testowego w języku Python w środowisku Google Colab. Zostanie opracowany program o budowie modułowej, w którym wyniki kolejnych etapów przetwarzania będą przechowywane i przekazywane pomiędzy modułami za pomocą plików pośrednich w formacie .pkl zapisywanych na dysku Google Drive.

Proces będzie obejmował: wybór i wczytanie zbioru danych, selekcję kolumn, ujednolicenie osi czasu do szeregu dziennego, czyszczenie danych, uzupełnianie braków, kodowanie zmiennych kategorycznych, skalowanie cech numerycznych oraz generowanie cech (cechy kalendarzowe, opóźnienia oraz statystyki kroczące). Następnie dane zostaną podzielone na zbiór treningowy i testowy.

W analizie zostaną uwzględnione dwa podejścia prognostyczne: metody tradycyjne (naive, moving average, simple exponential smoothing, ARIMA) oraz modele uczenia maszynowego (linear regression, ridge, lasso, random forest, SVM/SVR, XGBoost, MLP). Modele zostaną przetestowane w dwóch wariantach: statycznym (uczenie na zbiorze treningowym i prognoza dla całego okresu testowego) oraz kroczącym (rolling), w którym prognoza będzie aktualizowana krok po kroku w kolejnych dniach. Parametry modeli będą dobierane ręcznie oraz z wykorzystaniem przeszukiwania siatki (grid search) i przeszukiwania losowego (random search).

Wynikami pracy będą miary jakości prognozy (MAE, RMSE, MAPE, R^2) oraz rezultaty dodatkowego modułu symulacji kontroli zapasów, w którym na podstawie prognoz zostaną wyznaczone: liczba dni braku zapasu oraz średni poziom zapasu przy zadanych założeniach (początkowy zapas, poziom bezpieczeństwa, horyzont czasowy planowania zamówienia). Testy zostaną przeprowadzone na rzeczywistych zbiorach danych sprzedażowych pochodzących z platformy Kaggle [1]. Wybrane zostaną zbiory reprezentujące różne poziomy jakości danych, w tym dane zawierające braki, wysoki poziom zmienności, wiele cech opisujących sprzedaż oraz różny stopień uporządkowania. Umożliwi to ocenę odporności poszczególnych metod na niedoskonałości danych oraz analizę wpływu jakości danych na dokładność prognozowania oraz na wskaźniki kontroli zapasów, takie jak liczba dni braku zapasu oraz średni poziom zapasu.

3. Zarządzanie łańcuchem dostaw

Zarządzanie łańcuchem dostaw (ang. Supply Chain Management, SCM) to koordynacja procesów związanych z przepływem produktów i materiałów od dostawców do finalnych nabywców. Obejmuje etapy takie jak zaopatrzenie, produkcja i dystrybucja.

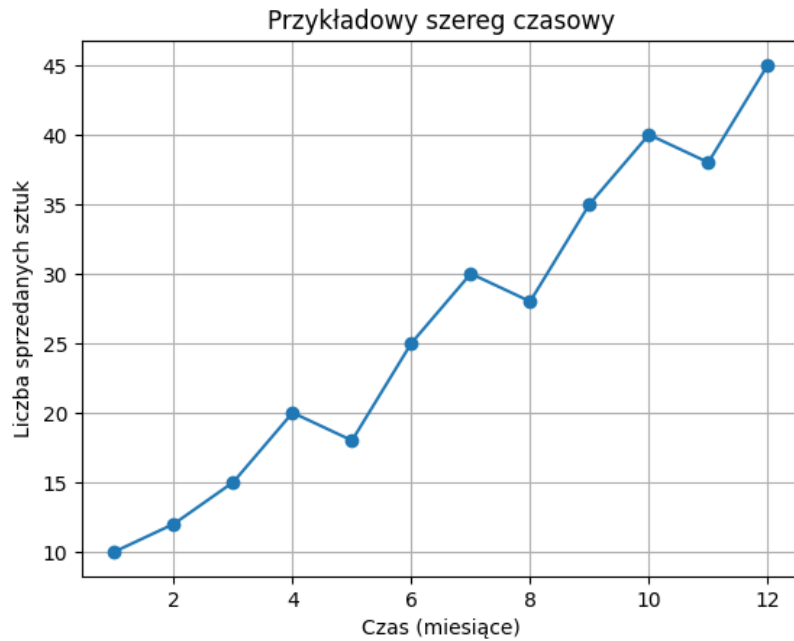
Zadania stawiane przed zarządzaniem łańcuchem dostaw

- Równoczesna minimalizacja kosztów związanych z logistyką (np. koszty transportu, magazynowania itp.) oraz zapewnienie realizacji zamówień klienta w zadanej ilości i określonym terminie.
- Skracanie czasu potrzebnego na poszczególne procesy, dążenie do zwiększenia elastyczności przy jednoczesnym zmniejszaniu wszelkich kosztów związanych z logistyką [2].

Elementy łańcucha dostaw:

- Planowanie wszelkich zasobów potrzebnych w łańcuchu dostaw (od dostaw poprzez produkcję aż do dystrybucji).
- Zaopatrzenie: wybór dostawców, zamawianie, kontrola dostaw, zarządzanie zapasami w magazynie.
- Produkcja obejmuje czynności związane z wytwarzaniem produktów, odpowiednie dostawy do miejsc wytwarzania i przetwarzania oraz odbiory w odpowiednim czasie i ilości. Zawiera także kontrolę ilościową procesów przed, w trakcie i po produkcji. Szczególnie istotna jest kontrola produkcji w toku, a nie tylko na wejściu i wyjściu procesu produkcji.
- Dostawa/dystrybucja to realizacja dostaw do klientów w odpowiednim czasie i ilości, zgodnie z zamówieniami klientów.
- Prognozowanie popytu jest procesem estymacji przyszłego zapotrzebowania na produkt na podstawie danych historycznych i stanowi istotny element minimalizacji kosztów logistycznych oraz zapewnienia realizacji zamówień klienta w wymaganej ilości i w wymaganym terminie. Tradycyjnie stosuje się do tego szeregi czasowe [3].

Szereg czasowy to ciąg uporządkowanych obserwacji/liczb, których dokonuje się w określonych (zwykle stałych) jednostkach czasu. Przykładem mogą być dane dotyczące sprzedaży produktów lub wielkości produkcji w poszczególnych miesiącach. W przypadku prognozowania szereg czasowy posiada dwa podstawowe elementy: trend i sezonowość. Trend to kierunek, w którym zmierza szereg czasowy w długim okresie czasu, np. wzrost, spadek lub stabilizacja. Sezonowość to pojawiające się zmiany ilościowe (wzrosty, spadki), powtarzalne w określonym czasie (np. w poszczególnych miesiącach roku).



Rysunek 3.1. Przykładowy szereg czasowy przedstawiający miesięczną sprzedaż produktu

Można wyróżnić wiele modeli szeregów czasowych stosowanych do prognozowania:

Tabela 3.1. Rodzaje modeli szeregów czasowych[4]

Typ szeregu czasowego	Modele szeregów czasowych
Szeregi stacjonarne	Naiwne Średnie ruchome Prosty model wygładzania wykładniczego Modele ARMA i ARIMA
Szeregi wykazujące trend	Wygładzanie wykładnicze Model liniowy Holta Modele analityczne Modele adaptacyjne
Szeregi wykazujące sezonowość	Model Wintersa Metoda wskaźnikowa Analiza harmoniczna

Przykładowe modele:

- **Metoda naiwna** lub średnia ruchoma to jedno z najprostszych modeli. Polegają na założeniu, że przyszłe wartości będą takie same jak przeszłe.

$$F_t = X_{t-1} \quad (1)$$

W metodzie naiwnej prognoza na okres t jest równa ostatniej zaobserwowanej wartości szeregu czasowego.

- **Średnia ruchoma**

$$F_t = \frac{1}{n} \sum_{i=1}^n X_{t-i} \quad (2)$$

Metoda średniej ruchomej polega na obliczaniu średniej z poprzednich n wartości w szeregu czasowym. Prognoza F_t na okres t jest średnią arytmetyczną z n ostatnich obserwacji $X_{t-1}, X_{t-2}, \dots, X_{t-n}$.

- **Wykładanie wykładnicze** stosuje się średnią ruchomą z przeszłych wartości szeregu, ale z różną wagą. Starsze dane mają niższą wagę, zaś nowsze wyższą przypisaną wagę.

$$F_t = F_{t-1} + \alpha(D_{t-1} - F_{t-1}) \quad (3)$$

Ten wzór reprezentuje klasyczną formę wykładania wykładniczego. Uwzględnia prognozę z poprzedniego okresu F_{t-1} oraz rzeczywistą wartość D_{t-1} z poprzedniego okresu, zaś F_t to prognoza z bieżącego okresu. Różnica między D_{t-1} a F_{t-1} jest korygowana współczynnikiem α , który kontroluje wpływ nowej wartości na bieżącą prognozę.

- **Model ARMA** (AutoRegressive Moving Average), który łączy AR (autoregresję), opisującą zależności bieżących wartości od przeszłych wartości (wskazuje, jak bardzo przeszłe wartości mają wpływ na wartość aktualną), oraz MA (średnią ruchomą błędów prognozy).
- **Model ARIMA** (AutoRegressive Integrated Moving Average) stanowi rozszerzenie modelu ARMA. Oprócz autoregresji oraz średniej ruchomej zawiera element różnicowania (I – Integrated), polegający na obliczaniu różnic między kolejnymi obserwacjami szeregu czasowego. Operacja ta ma na celu usunięcie trendu i stabilizację szeregu czasowego, tak aby jego podstawowe właściwości statystyczne, takie jak średnia oraz wariancja, nie zmieniały się w czasie.

$$F_t = c + \sum_{i=1}^p \phi_i \Delta^d X_{t-i} + \varepsilon_t + \sum_{j=1}^q \theta_j \varepsilon_{t-j} \quad (4)$$

gdzie:

- F_t – prognozowana wartość na okres t ,
- c – stała modelu,
- $\Delta^d X_{t-i}$ – wartości szeregu czasowego po d -krotnym różnicowaniu z i -tego poprzedniego okresu,
- ϕ_i – współczynniki części autoregresyjnej,
- ε_t – bieżący składnik losowy (błąd),
- ε_{t-j} – błędy prognozy z poprzednich okresów,
- θ_j – współczynniki części średniej ruchomej.

Model ARIMA (Autoregressive Integrated Moving Average) łączy w sobie trzy elementy: część autoregresyjną, różnicowanie oraz średnią ruchomą. Różnicowanie polega na obliczaniu różnic pomiędzy kolejnymi obserwacjami szeregu czasowego, co pozwala ograniczyć trend w danych, natomiast część autoregresyjna i średniej ruchomej opisują zależność bieżącej wartości od wcześniejszych obserwacji oraz wcześniejszych błędów prognozy.

- **Model Holta** to rozwinięcie wykładniczego, w którym do modelu dodaje się trend (dodane są parametry alfa i beta) [5].
- **Potrójne wykładnicze**, metoda Wintersa. W odróżnieniu od modelu Holta zawiera dodatkowo sezonowość. Trzy główne składniki tej metody to: średnia, trend i sezonowość [6].

Zarządzanie zapasami

Polega na ilościowym zarządzaniu produktami/materiałami składowanymi w magazynie. Główne cele to minimalizacja kosztów, takich jak koszty magazynowania i zamrożonego kapitału, przy jednoczesnym zapewnieniu realizacji zamówień klienta w wymaganej ilości oraz w wymaganym terminie. Tradycyjne metody, np. ABC, XYZ, EOQ [7], są statyczne i nie uwzględniają zmienności popytu. Najważniejsze punkty zarządzania zapasami to: ile jednostek zamówić, kiedy zamówić, które produkty/materiały są szczególnie istotne, oraz jak ograniczyć koszty.

Przykładowe metody:

- **Reguła 80/20**, 20 procent zapasów stanowi 80 procent ich wartości, co wymusza inne podejście do tak wartościowych zapasów.
- **Metoda ABC**, rozwinięcie reguły 80/20. Polega na podziale zapasów na trzy grupy: ważne, mniej ważne i nieważne (pod względem wartościowym).
- **Metoda XYZ**, jest uzupełnieniem metody ABC, ale zamiast wartości zapasów rozpatruje zmienność zapotrzebowania. Materiały dzielone są na trzy grupy: te ze stałym zapotrzebowaniem, te o umiarkowanej zmienności zapotrzebowania i występującymi wahaniami, te, na które zapotrzebowanie jest sporadyczne.
- **Modele (ABC i XYZ)** mogą być połączone, co pozwala na podział zapasów na 9 grup.
- **Model ekonomicznej wielkości zamówienia**, EOQ (Economic Order Quantity) oparty jest na zależności uwzględniającej koszty składania zamówienia, koszty przechowywania zapasów w magazynie oraz roczne zapotrzebowanie na dany materiał.
- **Metoda „dokładnie na czas” (Just-In-Time)** polega na systemie gospodarki zapasami, w którym niezbędne materiały wpływają dokładnie wtedy, kiedy są potrzebne. System ten zmniejsza przestrzeń magazynową potrzebną do przechowywania materiałów. Polega ona na zamawianiu materiałów w mniejszych partiach.

Ważną rolę w zarządzaniu zapasami odgrywają koszty. Główne składniki kosztów to: koszty zamawiania lub tworzenia zapasów, koszty magazynowania (trzymania na

magazynie) oraz koszty wynikające z braku zapasów w momencie, gdy są potrzebne [8]. Opis problemów, na które napotykamy w przypadku użycia tradycyjnych metod (opartych na szeregach czasowych):

- Nie radzą sobie z dużą liczbą czynników wpływających na zapotrzebowanie, takich jak sezonowość, trendy, promocje, zmienność popytu, czas dostawy czy czynniki zewnętrzne. Opierają się zazwyczaj na niewielkiej liczbie cech, głównie na wartościach historycznych w czasie, co ogranicza ich zdolność do dokładnego prognozowania.
- Często generują błąd zwany efektem bata. Efekt "bata" (ang. "bullwhip effect") jest zjawiskiem występującym w łańcuchu dostaw. Polega na zwiększającej się zmienności i niestabilności zamówień w miarę przesuwania się w górę łańcucha dostaw – od ostatecznego punktu sprzedaży do dostawcy surowców [9][10].

4. Uczenie maszynowe

4.1. Metody uczenia maszynowego i ich zastosowania

Uczeniem maszynowym nazywamy dziedzinę nauki (i sztukę) programowania komputerów w sposób umożliwiający im uczenie się z danych. Oto nieco ogólniejsza definicja: Uczenie maszynowe to dziedzina nauki dająca komputerom możliwość uczenia się bez konieczności ich jawnego programowania[11].

W obszarze uczenia maszynowego można wyodrębnić:

- **Uczenie nadzorowane** – model uczy się na podstawie danych treningowych, które zawierają dane wejściowe oraz przypisane do nich etykiety (rozwiązania, dane wyjściowe). Na tej podstawie model poznaje zależności między danymi wejściowymi a odpowiadającymi im wynikami, co pozwala mu przewidywać klasy lub wartości dla nowych, wcześniej niewidzianych danych.
- **Uczenie nienadzorowane** – dane uczące są nieoznakowane. Nie posiadają więc etykiet, model sam znajduje strukturę lub zależności na podstawie surowych danych, które mu przekazujemy.
- **Uczenie ze wzmocnieniem** – model wchodzi w interakcję ze środowiskiem. Nie przekazujemy gotowego zbioru danych, lecz model sam zbiera dane poprzez działanie w środowisku i uczy się na podstawie otrzymywanych nagród lub kar.
- **Uczenie półnadzorowane** – algorytmy uczą się na danych częściowo oznakowanych, czyli zawierających zarówno dane z etykietami, jak i bez etykiet[4][11][12].



Rysunek 4.1. Rodzaje uczenia maszynowego [13]

Modele uczenia maszynowego można zastosować do poszczególnych elementów łańcucha dostaw (prognozowania popytu, zarządzania zapasami).

Według Katherine Gail Nowadly i Sohyun Jung z Massachusetts Institute of Technology, najlepsze modele uczenia maszynowego stosowane w prognozowaniu popytu to las losowy, sieci neuronowe, regresja liniowa oraz maszyna wektorów nośnych (SVM) [14]. Z kolei Real Carbonneau, Kevin Laframboise oraz Rustam Vahidov w artykule pt. „Application of Machine Learning Techniques for Supply Chain Demand Forecasting”, opublikowanym w czasopiśmie *European Journal of Operational Research*, wskazują rekurencyjne sieci neuronowe oraz SVM jako metody szczególnie odpowiednie do prognozowania popytu [15]. Natomiast J. Shahrabi, S.S. Mousavi oraz M. Heydar w artykule pt. „Supply Chain Demand Forecasting: A Comparison of Machine Learning Techniques and Traditional Methods” również wskazują SVM oraz sieci neuronowe jako jedne z najbardziej efektywnych metod [16]. Według Ritsaarta Bergsmy, Corné de Ruijtera oraz Sandjai Bhulaia z Wydziału Matematyki Uniwersytetu VU w Amsterdamie oraz z Katedry Finansów Łańcucha Dostaw Uniwersytetu Nauk Stosowanych Windesheim, w kontekście zarządzania zapasami szczególnie istotne zastosowanie znajdują metody Q-learning oraz RNN [17]. Z kolei Jinjin Li, Huijun Huang, Yuping Gong, Lei Wang, Xiangui Yin oraz Yichang Liu w artykule „A Review of Artificial Intelligence in Inventory Management: Methods, Applications and Directions”, opublikowanym w czasopiśmie *International Journal of Advanced Computer Science and Applications*, wskazują, że w zarządzaniu zapasami dominujące zastosowanie znajdują sieci neuronowe, a dodatkowo wykorzystywane są metody takie jak k-średnich oraz XGBoost [18].

Poniżej przedstawiono wybrane modele uczenia maszynowego stosowane w prognozowaniu popytu i zarządzaniu zapasami:

- **Neural Networks (NN)** – zwane również sztucznymi sieciami neuronowymi, są modelami inspirowanymi strukturą biologicznych neuronów. Składają się z połączonych ze sobą neuronów zorganizowanych w warstwy, które przetwarzają dane wejściowe i przekazują informacje między sobą za pomocą wag, umożliwiając modelowanie złożonych zależności.
- **Recurrent Neural Networks (RNN)** – są szczególnym rodzajem sieci neuronowych posiadających połączenia zwrotne (rekurencyjne), które umożliwiają wykorzystywanie informacji z poprzednich kroków czasowych. Dzięki temu mogą analizować dane sekwencyjne, takie jak szeregi czasowe, co czyni je szczególnie przydatnymi w prognozowaniu popytu. Proces uczenia sieci RNN realizowany jest z wykorzystaniem wstecznej propagacji błędów w czasie (ang. backpropagation through time, BPTT). Metoda ta polega na dostosowywaniu wag sieci na podstawie błędów prognoz obliczanych w kolejnych krokach czasowych, co umożliwia minimalizację funkcji straty oraz poprawę dokładności modelu.
- **Support Vector Machines (SVM)** – są stosowane zarówno do klasyfikacji, jak i regresji (w tej wersji jako Support Vector Regression, SVR). Model wyznacza hiperpowierzchnię (czyli granicę decyzyjną), która najlepiej rozdziela dane lub opisuje zależność między zmiennymi. Metoda ta jest skuteczna w przypadku danych o dużej liczbie cech i może być wykorzystywana do prognozowania popytu lub zapotrzebowania materiałowego.
- **Drzewa decyzyjne** – są modelami wykorzystywanymi do klasyfikacji i regresji, które tworzą strukturę hierarchiczną w postaci drzewa. Pozwalają na identyfikację najważniejszych cech wpływających na prognozowaną wartość, co umożliwia analizę czynników wpływających na popyt oraz jego prognozowanie.
- **Support Vector Regression (SVR)** jest odmianą metody Support Vector Machines stosowaną do rozwiązywania problemów regresji, czyli przewidywania wartości liczbowych. W odróżnieniu od klasyfikacyjnej wersji SVM, która przypisuje dane do określonych klas, SVR przewiduje konkretną wartość liczbową na podstawie danych wejściowych. [19].
- **Q-learning** jest algorytmem uczenia ze wzmocnieniem stosowanym do wyznaczania strategii działania w problemach decyzyjnych. Algorytm polega na uczeniu agenta podejmowania decyzji poprzez interakcję ze środowiskiem oraz aktualizację wartości funkcji Q na podstawie otrzymywanych nagród. Celem procesu uczenia jest maksymalizacja skumulowanej nagrody w długim horyzoncie czasowym. Działanie algorytmu Q-learning obejmuje:
 - inicjalizację tablicy wartości Q,
 - wybór oraz wykonanie akcji,
 - obserwację nagrody oraz nowego stanu,
 - aktualizację wartości funkcji Q.

W kontekście zarządzania zapasami Q-learning może być wykorzystywany do podejmowania decyzji dotyczących wielkości oraz momentu składania zamówień [20].

Obszary zastosowań algorytmów uczenia maszynowego:

Algorytmy uczenia maszynowego można podzielić na trzy główne grupy: uczenie nadzorowane, uczenie nienadzorowane oraz uczenie ze wzmocnieniem.

Uczenie nadzorowane znajduje zastosowanie w problemach regresji oraz klasyfikacji.

W przypadku regresji (prognozowania wartości liczbowych) stosowane są m.in.: K-Nearest Neighbors Regression, Regression Tree, Linear Regression, XGboost.

W problemach klasyfikacji wykorzystywane są m.in.: K-Nearest Neighbors (KNN), Naive Bayes, Random Forest.

Uczenie nadzorowane jest metodą uczenia maszynowego, w której model jest trenowany na podstawie danych wejściowych wraz z przypisanymi do nich poprawnymi wynikami (tzw. etykietami). Na tej podstawie model uczy się zależności między zmiennymi i może następnie przewidywać wyniki dla nowych danych.

Uczenie nadzorowane znajduje zastosowanie w problemach regresji oraz klasyfikacji.

W przypadku regresji (prognozowania wartości liczbowych) stosowane są m.in.: K-Nearest Neighbors Regression, Regression Tree, Linear Regression, XGBoost.

W problemach klasyfikacji wykorzystywane są m.in.: K-Nearest Neighbors (KNN), Naive Bayes, Random Forest.

Uczenie nienadzorowane jest metodą uczenia maszynowego, w której model analizuje dane bez wcześniej przypisanych etykiet lub poprawnych wyników. Celem jest znalezienie ukrytych wzorców lub zależności w danych.

Uczenie nienadzorowane stosowane jest przede wszystkim w analizie skupień oraz wykrywaniu anomalii.

Do analizy skupień należą m.in. metoda k-średnich, hierarchiczna analiza skupień oraz DBSCAN.

W wykrywaniu anomalii wykorzystywane są m.in. jednoklasowa maszyna wektorów nośnych, analiza głównych składowych (PCA) oraz jądrowa analiza głównych składowych [12], [21].

Uczenie ze wzmocnieniem jest metodą uczenia maszynowego, w której model (agent) uczy się poprzez interakcję ze środowiskiem, otrzymując nagrody lub kary za podejmowane działania. Na podstawie tych informacji model stopniowo uczy się strategii prowadzącej do maksymalizacji uzyskiwanej nagrody.

Uczenie ze wzmocnieniem znajduje zastosowanie w problemach sekwencyjnego podejmowania decyzji, w których model uczy się strategii działania na podstawie interakcji ze środowiskiem [4][22].

Uczenie głębokie obejmuje modele oparte na wielowarstwowych sieciach neuronowych, wykorzystywane w zadaniach takich jak rozpoznawanie obrazów, przetwarzanie języka naturalnego (NLP), analiza sygnałów oraz prognozowanie wartości liczbowych [4][23].

4.2. Wybrane algorytmy uczenia maszynowego

Klastrowanie metoda uczenia nienadzorowanego (Unsupervised Learning)

Polega na dzieleniu zbioru danych na grupy (skupienia) na podstawie podobieństwa między jego elementami. Termin "clustering" jest szeroko używany i odnosi się do procesu tworzenia grup (klas) obiektów na podstawie pewnych kryteriów, takich jak odległość między obiektami. Celem jest stworzenie skupisk podobnych obiektów wewnątrz klastra, które różnią się od obiektów w innych klastrach.

Użycie algorytmów klastrowania w uczeniu nienadzorowanym polega na przekazaniu na wejście algorytmu dużej ilości danych, które nie są oznaczone. Algorytm grupuje je na podstawie podobieństw [24].

Algorytm grupowania K-średnich (K-means) Jest to algorytm oparty na centroidach. W iteracyjny sposób punkty są przypisywane do grup. Celem jest minimalizacja odległości między punktami a danym środkiem (centroidem) [24].

Działanie algorytmu k-średnich obejmuje następujące etapy:

- wybór liczby klastrów,
- inicjalizację centroidów, np. w sposób losowy,
- przypisanie każdego punktu danych do najbliższego klastra na podstawie obliczonej odległości od centroidu,
- aktualizację centroidów poprzez wyznaczenie średniej wartości punktów należących do danego klastra,
- powtarzanie procedury przypisania i aktualizacji do momentu ustabilizowania się przyporządkowania punktów do klastrów [24].

Support Vector Regression (SVR) – algorytm uczenia nadzorowanego Jest metodą regresji wywodzącą się z algorytmu Support Vector Machines (SVM). W przeciwieństwie do klasyfikacyjnej wersji SVM, która przypisuje dane do klas, SVR służy do przewidywania wartości liczbowych.

Model SVR wyznacza funkcję regresji w postaci:

$$f(x) = w \cdot x + b \quad (5)$$

gdzie: w – wektor wag modelu, b – wyraz wolny, x_i – i -ta obserwacja danych wejściowych, y_i – rzeczywista wartość wyjściowa odpowiadająca obserwacji x_i .

W algorytmie SVR dopasowanie modelu jest realizowane poprzez minimalizację funkcji, która opisuje błąd predykcji. W tym celu wprowadzany jest margines tolerancji ϵ , określający dopuszczalny poziom błędów.

Jeżeli dla danej obserwacji spełniony jest warunek:

$$|y_i - (w \cdot x_i + b)| \leq \epsilon \quad (6)$$

to błąd tej obserwacji jest traktowany jako akceptowalny i nie jest uwzględniany w obliczaniu błędu modelu.

Jeżeli natomiast błąd przekracza wartość ϵ , uwzględniana jest jedynie nadwyżka ponad margines:

$$|y_i - (w \cdot x_i + b)| - \epsilon \quad (7)$$

Ponieważ kara nie może być ujemna, stosuje się funkcję:

$$\max(0, |y_i - (w \cdot x_i + b)| - \epsilon) \quad (8)$$

Wyrażenie to określa błąd pojedynczej obserwacji. Sumując je dla wszystkich obserwacji w zbiorze treningowym, otrzymujemy łączny błąd modelu:

$$\sum \max(0, |y_i - (w \cdot x_i + b)| - \epsilon) \quad (9)$$

Aby dodatkowo ograniczyć nadmierne dopasowanie modelu do danych, do funkcji kosztu wprowadzany jest składnik zależny od wag modelu:

$$\frac{1}{2} \|w\|^2 \quad (10)$$

Ostatecznie proces uczenia polega na minimalizacji funkcji:

$$\frac{1}{2} \|w\|^2 + C \sum \max(0, |y_i - (w \cdot x_i + b)| - \epsilon) \quad (11)$$

Parametr C określa, jak silnie błędy przekraczające margines ϵ wpływają na końcowe dopasowanie modelu [11][19][25].

Algorytm Regression Tree Drzewo regresji jest modelem wykorzystywanym do przewidywania wartości numerycznych. Każdy węzeł wewnętrzny reprezentuje test na jednej z cech danych, natomiast liść zawiera przewidywaną wartość liczbową.

Rozważmy przykład predykcji ceny domów na podstawie ich cech (np. powierzchni, liczby pokoi, rodzaju budownictwa). Zbiór uczący przedstawiono w tabeli 4.1, gdzie x_1, x_2 oznaczają cechy wejściowe, a y cenę domu.

Celem algorytmu jest podział danych na grupy w taki sposób, aby zmienność wartości y w każdej grupie była możliwie najmniejsza.

Działanie algorytmu można opisać następująco:

- w węźle początkowym (korzeniu) analizowane są wszystkie możliwe podziały danych według dostępnych cech, np. „Czy powierzchnia domu jest mniejsza lub równa 150 m²?”,
- dla każdego możliwego podziału dane dzielone są na dwa zbiory, np. R_0 oraz R_1 ,
- dla każdej z powstałych grup (np. R_0 i R_1) obliczana jest miara błędu. Najczęściej sto-

Tabela 4.1. Dane uczące

L.p.	x		y
	x_1	x_2	
1	5.5	6.0	123
2	-4.5	-7.0	-211
$\vdots$	$\vdots$	$\vdots$	$\vdots$
N	0.5	8.0	222

suje się sumę kwadratów różnic między rzeczywistą wartością y_i a średnią wartością $\bar{y}$ w danej grupie:

$$\sum (y_i - \bar{y})^2 \quad (12)$$

gdzie $\bar{y}$ oznacza średnią wartość zmiennej y w analizowanej grupie.

- wybierany jest taki podział, który minimalizuje łączny błąd w obu grupach,
- dla każdego nowego węzła procedura jest powtarzana rekurencyjnie — możliwe są kolejne pytania, np. „Czy dom jest nowy czy stary?”, „Ile ma pokoi?”,
- w węzłach końcowych (liściach) jako prognozę przyjmuje się średnią wartość ceny domów w danym obszarze.

Podział drzewa kończy się, gdy spełniony zostanie jeden z warunków:

- liczba obserwacji w węźle spadnie poniżej ustalonego progu,
- dalszy podział nie powoduje istotnej redukcji błędu,
- osiągnięta zostanie maksymalna głębokość drzewa[26].

Algorytm K-nearest neighbors (KNN) Służy do klasyfikacji danych na podstawie podobieństwa do innych obserwacji w zbiorze treningowym. Działanie algorytmu można przedstawić w następujących krokach:

- wybierana jest liczba sąsiadów K , która określa, ile najbliższych obserwacji zostanie uwzględnionych podczas klasyfikacji,
- dla nowej obserwacji obliczane są odległości pomiędzy nią a wszystkimi punktami w zbiorze treningowym,
- obliczone odległości są porządkowane od najmniejszej do największej,
- wybieranych jest K najbliższych sąsiadów nowej obserwacji,
- nowa obserwacja zostaje przypisana do klasy, która występuje najczęściej wśród wybranych K sąsiadów,
- przykładowo, gdy $K = 3$, nowy punkt zostaje przypisany do klasy dominującej wśród trzech najbliższych sąsiadów.

Jest to bardzo prosty sposób przypisania danych do klas. [4][27].

Algorytm Support Vector Machines (SVM) Służy do rozwiązywania problemów klasyfikacyjnych poprzez wyznaczenie hiperpłaszczyzny rozdzielającej dane należące do różnych klas.

Działanie algorytmu można przedstawić w następujących krokach:

- dane treningowe reprezentowane są jako punkty w przestrzeni n -wymiarowej,
- wyznaczana jest hiperpłaszczyzna, czyli granica rozdzielająca dane należące do różnych klas (w przestrzeni dwuwymiarowej jest to linia, w trójwymiarowej płaszczyzna), celem jest znalezienie takiej granicy, która maksymalizuje margines, czyli odległość między tą granicą a najbliższymi punktami obu klas,
- próbki znajdujące się najbliżej wyznaczonej hiperpłaszczyzny nazywane są wektorami nośnymi. To ich położenie wyznacza szerokość marginesu oraz ostateczne położenie hiperpłaszczyzny.
- w przypadku klasyfikacji z twardym marginesem wymaga się, aby wszystkie próbki były poprawnie rozdzielone i znajdowały się poza marginesem,
- w klasyfikacji z miękkim marginesem dopuszcza się naruszenia marginesu; parametr C kontroluje kompromis pomiędzy szerokością marginesu a liczbą dopuszczalnych błędów,
- jeżeli dane są liniowo separowalne, stosuje się liniowy SVM, w którym hiperpłaszczyzna jest linią (w 2D),
- jeżeli dane nie są liniowo separowalne, stosuje się nieliniowy SVM; dane są wówczas odwzorowywane do przestrzeni o wyższym wymiarze, w której możliwe jest liniowe rozdzielanie klas,
- odwzorowanie to realizowane jest przy użyciu funkcji jądrowych (kernel). Mechanizm ten, nazywany sztuczką z funkcją jądra (kernel trick), pozwala modelowi rozdzielać dane nieliniowe poprzez zastosowanie odpowiedniej funkcji jądra, która umożliwia wyznaczenie nieliniowej granicy między klasami. W kolejnych punktach przedstawiono przykładowe funkcje jądra stosowane w algorytmie SVM.
- przykładem funkcji jądra jest jądro wielomianowe stopnia 2:

$$k(x, y) = (x^T y)^2 \quad (13)$$

gdzie $x = (x_1, x_2, \dots, x_n)$ oraz $y = (y_1, y_2, \dots, y_n)$ oznaczają wektory cech dwóch obserwacji danych. Symbol T oznacza transpozycję wektora x , czyli zmianę wektora kolumnowego na wierszowy.

Wówczas wyrażenie $x^T y$ oznacza sumę iloczynów odpowiadających sobie cech:

$$x^T y = x_1 y_1 + x_2 y_2 + \dots + x_n y_n \quad (14)$$

Funkcja ta odpowiada sytuacji, w której do danych zostałyby dodane nowe cechy utworzone z istniejących zmiennych.

- przykładem funkcji jądra jest funkcja bazowa radialna (RBF). Jej wzór:

$$k(\mathbf{x}, \mathbf{y}) = e^{-\gamma \|\mathbf{x} - \mathbf{y}\|^2} \quad (15)$$

gdzie $\mathbf{x}$ oraz $\mathbf{y}$ oznaczają wektory cech dwóch obserwacji danych, $\|\mathbf{x} - \mathbf{y}\|$ oznacza odległość euklidesową między tymi punktami, γ jest parametrem modelu określającym, jak szybko maleje wartość funkcji jądra wraz ze wzrostem odległości między punktami. Duża wartość γ powoduje, że wartość funkcji jądra bardzo szybko maleje wraz ze wzrostem odległości między punktami, dlatego na wynik modelu wpływają głównie bardzo bliskie obserwacje. Mała wartość γ powoduje wolniejszy spadek wartości funkcji jądra, dzięki czemu na wynik modelu mogą wpływać także bardziej odległe obserwacje. Natomiast e jest stałą matematyczną (podstawą logarytmu naturalnego).

Wyrażenie

$$\|\mathbf{x} - \mathbf{y}\|^2 \quad (16)$$

oznacza kwadrat odległości euklidesowej między wektorami $\mathbf{x}$ i $\mathbf{y}$.

Jądro RBF mierzy podobieństwo między dwoma punktami. Gdy odległość $\|\mathbf{x} - \mathbf{y}\|$ jest mała, wartość funkcji jądra jest bliska 1, natomiast gdy odległość rośnie, wartość funkcji zbliża się do 0.

Przykładowo rozważmy dwa punkty: $x_1 = -3$ oraz $x_1 = 2$. Dla parametru $\gamma = 0.5$ można obliczyć nowe wartości cech.

Dla próbki $x_1 = 2$ odległość od pierwszego punktu wynosi 1, natomiast od drugiego punktu wynosi 3. Wówczas nowe cechy przyjmują wartości:

$$x_2 = \exp(-0.5 \times 1^2) \approx 0.61 \quad (17)$$

$$x_3 = \exp(-0.5 \times 3^2) \approx 0.01 \quad (18)$$

- ponieważ funkcja jądra RBF określa podobieństwo punktów na podstawie ich odległości, algorytm SVM może budować granicę klasyfikacji tak, aby punkty podobne do siebie należały do tej samej klasy. Ponieważ podobieństwo zależy od odległości w wielu miejscach przestrzeni, granica między klasami może przyjmować bardziej zakrzywiony kształt[4][11][28].

5. Dobór narzędzi i przygotowanie środowiska analitycznego

Metody uczenia maszynowego i tradycyjne zastosowane w testach W pierwszej kolejności dokonano doboru metod prognozowania popytu oraz metod wykorzystywanych w zarządzaniu zapasami. W poprzednim rozdziale wskazano, że do prognozowania popytu szczególnie często stosowane są metody uczenia maszynowego, takie jak: **regresja liniowa**, **las losowy**, **sieci neuronowe**, **SVM** oraz **XGBoost** (gradient boosting oparty na sekwencyjnym budowaniu drzew decyzyjnych). Ponieważ w analizowanym problemie przewidywane są wartości liczbowe, zastosowano odmiany tych metod przeznaczone do regresji. W związku z tym zamiast klasyfikacyjnej wersji SVM wykorzystano **Support Vector Regression (SVR)**, natomiast zamiast klasycznego lasu losowego zastosowano jego wariant regresyjny, czyli **Random Forest Regression**. Dodatkowo wykorzystano rozszerzenia regresji liniowej w postaci metod **Ridge** oraz **Lasso**. Metody te stanowią rozwinięcie regresji liniowej poprzez wprowadzenie regularyzacji, która ogranicza nadmierny wpływ zmiennych wejściowych na wynik modelu oraz pomaga zapobiegać przeuczeniu.

Odnosnie metod tradycyjnych, w poprzednim rozdziale wymieniono takie metody jak: **metoda naiwna**, **średnia ruchoma**, **Simple Exponential Smoothing** (prosta metoda wygładzania wykładniczego) oraz **ARIMA**. Właśnie te metody zostały wykorzystane w przeprowadzonych testach i posłużyły jako punkt odniesienia do porównania z metodami uczenia maszynowego.

Wybór języka programowania Na podstawie zestawień popularności języków programowania, takich jak TIOBE Index oraz PYPL (Popularity of Programming Language Index), do najczęściej wykorzystywanych języków programowania zalicza się m.in. **Python**, **JavaScript**, **Java** oraz **C++**. TIOBE Index jest rankingiem popularności języków programowania opartym na analizie liczby wyników wyszukiwania w popularnych wyszukiwarkach internetowych oraz liczby stron internetowych związanych z danym językiem. Z kolei PYPL jest indeksem popularności języków programowania opartym na analizie częstotliwości wyszukiwań w wyszukiwarce Google związanych z nauką poszczególnych języków programowania [29].

Na podstawie analizy artykułów dotyczących języków wykorzystywanych w uczeniu maszynowym można wskazać kilka technologii, które są szczególnie często stosowane w projektach tego typu, między innymi Python, R, Java, C++, Julia, Scala, JavaScript, Lisp, Haskell oraz Go [30], [31], [32].

- **Python** jest często wykorzystywany w uczeniu maszynowym ze względu na prostą składnię oraz szeroki ekosystem bibliotek wspierających analizę danych i budowę modeli, takich jak TensorFlow, Keras, NumPy czy Scikit-learn. Zaletą jest łatwość nauki oraz duża społeczność użytkowników i dostępność narzędzi wspierających

rozwój modeli ML [30]. Wadą jest niższa wydajność w porównaniu z językami takimi jak C lub C++ [32].

- **R** jest wykorzystywany w analizie statystycznej oraz eksploracji danych, co sprawia, że znajduje zastosowanie w projektach data science i uczenia maszynowego. Jego zaletą są rozbudowane narzędzia do analizy statystycznej i wizualizacji danych [31]. Wadą jest mniejsza wydajność [32].
- **Java** jest wykorzystywana w projektach uczenia maszynowego szczególnie w środowiskach korporacyjnych, gdzie modele muszą być integrowane z istniejącą infrastrukturą systemów informatycznych oraz dużymi systemami przetwarzania danych. Do zalet należy możliwość obsługi aplikacji o dużej skali, wysoka przepustowość oraz rozbudowany ekosystem narzędzi i bibliotek programistycznych [32]. Wadą jest mniejsza liczba bibliotek związanych bezpośrednio z uczeniem maszynowym w porównaniu z Pythonem oraz bardziej rozbudowaną i skomplikowaną składnią języka [32].
- **C++** jest wykorzystywany w projektach uczenia maszynowego wymagających wysokiej wydajności obliczeniowej oraz bezpośredniej kontroli nad zasobami systemowymi. Do jego głównych zalet należy wysoka szybkość działania oraz możliwość precyzyjnego zarządzania pamięcią, co pozwala optymalizować algorytmy i wykorzystywać sprzęt na niskim poziomie [31]. C++ jest szczególnie przydatny w aplikacjach czasu rzeczywistego, systemach robotycznych, przetwarzaniu obrazu. Wśród wad wskazuje się większą złożoność składni. Dodatkowo dostępność wysoko-poziomowych bibliotek ML jest mniejsza niż w przypadku Pythona [32].

Analiza popularności języków programowania w uczeniu maszynowym i data science wskazuje wyraźną dominację kilku technologii. Dane branżowe oraz ankiety przeprowadzone wśród specjalistów pokazują, że najczęściej wykorzystywane są przede wszystkim Python, SQL, R, C++ i Java [33], [34]. Na podstawie danych udostępnionych przez platformę Kaggle, zebranych wśród kilkudziesięciu tysięcy specjalistów zajmujących się analizą danych, wynika, że Python jest zdecydowanie najczęściej wykorzystywanym językiem programowania w obszarze data science i uczenia maszynowego [33]. W wielu analizach ankiety wskazuje się, że ponad 80% respondentów korzysta z tego języka w swojej pracy [34]. Popularność Pythona wynika przede wszystkim z dużej liczby bibliotek uczenia maszynowego oraz łatwości pracy z danymi [35].

Jedną z najważniejszych zalet języka Python w uczeniu maszynowym jest bardzo rozbudowany ekosystem bibliotek, które znacząco upraszczają implementację algorytmów oraz analizę danych [36][37]. Dzięki nim wiele złożonych operacji matematycznych, statystycznych oraz procesów trenowania modeli może być realizowanych przy użyciu gotowych funkcji, co przyspiesza rozwój projektów ML [38].

Do najważniejszych bibliotek Pythona wykorzystywanych w uczeniu maszynowym należą między innymi NumPy, Pandas, Scikit-learn oraz TensorFlow [36].

- **NumPy** – biblioteka wykorzystywana do obliczeń numerycznych oraz operacji na wielowymiarowych tablicach danych. Umożliwia wykonywanie operacji matematycznych oraz przetwarzanie dużych zbiorów danych numerycznych [38].
- **Pandas** – biblioteka przeznaczona do manipulacji oraz analizy danych. Udostępnia struktury danych takie jak *DataFrame*, które pozwalają w łatwy sposób filtrować, przekształcać oraz agregować dane wykorzystywane w procesie trenowania modeli [38].
- **Scikit-learn** – jedna z najczęściej wykorzystywanych bibliotek uczenia maszynowego w Pythonie. Zawiera wiele gotowych algorytmów, takich jak klasyfikacja, regresja czy klasteryzacja, co pozwala szybko budować i testować modele ML bez konieczności implementowania algorytmów od podstaw [36].
- **TensorFlow** – biblioteka wykorzystywana głównie w głębokim uczeniu (*deep learning*). Umożliwia budowanie i trenowanie sieci neuronowych oraz wykonywanie obliczeń na dużych zbiorach danych [36].

Biblioteki te współpracują ze sobą w ramach jednego ekosystemu. Na przykład NumPy i Pandas są wykorzystywane do przygotowania danych, natomiast Scikit-learn lub TensorFlow do trenowania modeli uczenia maszynowego [38].

Na podstawie przedstawionej analizy popularności języków programowania, dostępności narzędzi oraz ekosystemu bibliotek wspierających uczenie maszynowe, w niniejszej pracy zdecydowano się na wykorzystanie języka **Python**.

Wybór środowiska obliczeniowego Do realizacji eksperymentów wykorzystano środowisko **Google Colab**. Jest to platforma chmurowa umożliwiająca uruchamianie kodu w języku Python z wykorzystaniem zasobów obliczeniowych udostępnianych przez Google, do których dostęp odbywa się za pośrednictwem przeglądarki internetowej w formie notebooków podobnych do środowiska Jupyter Notebook [39]. Zaletą Google Colab jest brak konieczności instalacji i konfiguracji środowiska programistycznego na lokalnym komputerze. Użytkownik może rozpocząć pracę natychmiast po uruchomieniu notebooka, ponieważ większość bibliotek wykorzystywanych w uczeniu maszynowym jest już zainstalowana [40]. Istotną zaletą tego środowiska jest również dostęp do zasobów obliczeniowych w chmurze. Google Colab umożliwia także łatwe udostępnianie notebooków oraz współpracę wielu użytkowników nad tym samym projektem [39]. Dzięki tym właściwościom jest często wykorzystywany w projektach data science oraz uczenia maszynowego.

Architektura programu testowego i przebieg przetwarzania danych Program testowy został zaimplementowany w języku Python i uruchamiany w środowisku Google Colab. Całość została zaprojektowana w postaci architektury modułowej, w której każdy etap jest realizowany przez osobny moduł (plik .py). Moduły te są uruchamiane sekwencyjnie z poziomu głównego pliku sterującego. W zależności od scenariusza analitycznego możliwe

jest zastąpienie wybranych modułów innymi implementacjami, na przykład rozszerzenie modułu oceny prognoz o symulację zarządzania zapasami.

- **Środowisko uruchomieniowe i organizacja plików.** Program działa w środowisku Google Colab, natomiast wszystkie dane robocze oraz wyniki pośrednie są zapisywane na dysku Google Drive. W pierwszym kroku następuje zamontowanie dysku użytkownika, a następnie utworzenie katalogu projektu `ml_project`. Katalog ten zawiera wszystkie pliki wejściowe, dane pośrednie oraz wyniki eksperymentów. Dodatkowo ścieżka katalogu projektu jest dodawana do listy ścieżek systemowych Pythona (`sys.path`), co umożliwia bezpośrednie importowanie modułów projektu zapisanych na dysku Google Drive. Na początku programu wykonywany jest również restart procesu Pythona przy użyciu polecenia `os.kill(os.getpid(), 9)`, co pozwala rozpocząć pracę w czystym środowisku wykonawczym.
- **Sposób wymiany danych pomiędzy modułami.** Poszczególne moduły komunikują się ze sobą poprzez pliki pośrednie zapisywane w katalogu projektu. Dane tabelaryczne są zapisywane głównie w formacie `.pkl`, który pozwala na szybką serializację obiektów `pandas.DataFrame`. Serializacja jest procesem polegającym na przekształceniu obiektu znajdującego się w pamięci programu do postaci binarnej, która może zostać zapisana w pliku i później ponownie odtworzona w identycznej formie. W języku Python do tego celu wykorzystywany jest między innymi moduł `pickle`, który umożliwia zapisywanie i odczytywanie obiektów takich jak listy, słowniki, modele uczenia maszynowego. Konfiguracje wybierane przez użytkownika w interfejsie programu zapisywane są natomiast w plikach tekstowych `.txt`.
- **Wczytanie i przygotowanie danych wejściowych.** Proces rozpoczyna się od wyboru źródła danych. Użytkownik może wskazać plik w formacie CSV, plik JSON lub źródło API. Dane są zapisywane do pliku i następnie są wczytywane do obiektu `DataFrame` i zapisywane w pliku `df.pkl`.
- **Selekcja kolumn i identyfikacja typów danych.** W kolejnym etapie użytkownik wybiera kolumny przeznaczone do dalszej analizy, a następnie dane są filtrowane i zapisywane jako nowy zbiór danych. Kolejny moduł analizuje typy danych w wybranych kolumnach i klasyfikuje je jako zmienne numeryczne, kategoriowe lub datowe.
- **Normalizacja osi czasu.** Użytkownik wybiera kolumnę reprezentującą datę oraz sposób agregacji duplikatów dat (np. suma lub średnia dla kolumn numerycznych). Program tworzy ciągłą dzienną oś czasu pomiędzy minimalną a maksymalną datą w zbiorze danych. Brakujące dni są uzupełniane zgodnie z wybraną metodą, np. poprzez wstawienie wartości zero, zastosowanie metody `forward fill` (czyli uzupełniania brakujących wartości ostatnią dostępną obserwacją) lub uzupełnienie średnią.
- **Czyszczenie danych.** Kolejnym etapem jest obsługa brakujących wartości w danych. Program prezentuje użytkownikowi informację o liczbie braków w każdej kolumnie i

pozwala wybrać odpowiednią metodę ich obsługi. Dostępne są między innymi opcje usunięcia wierszy z brakami, uzupełnienia wartości zerem, uzupełnienia średnią lub pozostawienia wartości bez zmian.

- **Inżynieria cech.** Generowane są dodatkowe cechy opisujące strukturę czasową danych.
 - tworzenie cech kalendarzowych takich jak rok, miesiąc, dzień miesiąca oraz dzień tygodnia,
 - generowanie cech opóźnionych (lagów) opisujących wartości zmiennej w poprzednich okresach,
 - obliczanie statystyk kroczących, np. średniej kroczącej dla wybranego okna czasowego.
- **Kodowanie zmiennych kategoriycznych.** Zmienne kategoriyczne to zmienne opisujące cechy jakościowe, których wartości reprezentują różne kategorie lub klasy, a nie wielkości liczbowe. Ponieważ większość algorytmów uczenia maszynowego wymaga danych w postaci numerycznej, zmienne kategoriyczne muszą zostać przekształcone do formy liczbowej. W tym celu zastosowano dwie metody kodowania:
 - kodowanie one-hot polegające na utworzeniu zmiennych binarnych dla każdej kategorii,
 - kodowanie średnią (mean encoding), w którym każdej kategorii przypisywana jest liczba obliczona na podstawie danych historycznych. Wartość ta jest średnią wartością zmiennej, którą model ma przewidywać, obliczoną dla wszystkich obserwacji należących do danej kategorii.
- **Wybór zmiennej docelowej i przygotowanie danych do uczenia.** Użytkownik wybiera jedną z kolumn jako zmienną docelową. Dane są następnie dzielone na macierz cech X oraz zmienną docelową y . Następnie wybrane kolumny numeryczne mogą zostać poddane standaryzacji przy użyciu metody `StandardScaler`. Standaryzacja polega na przekształceniu wartości zmiennych tak, aby miały średnią równą 0 oraz odchylenie standardowe równe 1. Zmienne mogą mieć bardzo różne zakresy wartości (np. liczba sprzedanych produktów, ceny), co może wpływać na działanie niektórych algorytmów uczenia maszynowego. Skalowanie danych pozwala ograniczyć wpływ różnic skali i ułatwia modelom prawidłowe uczenie się.
- **Podział danych na zbiór treningowy i testowy.** Użytkownik określa proporcję zbioru testowego, na podstawie której dane są dzielone na cztery zbiory: X_{train} , X_{test} , y_{train} oraz y_{test} .
- **Konfiguracja modeli.** Program umożliwia wybór dwóch grup modeli:
 - tradycyjnych metod prognozowania, takich jak metoda naiwna, średnia ruchoma, proste wygładzanie wykładnicze oraz ARIMA,
 - modeli uczenia maszynowego, takich jak regresja liniowa, Ridge, Lasso, Random Forest, SVR, XGBoost oraz MLP.

Dla modeli tradycyjnych użytkownik wybiera typ modelu oraz jego podstawowe parametry, na przykład długość okna dla średniej ruchomej, parametr wygładzania dla metody wygładzania wykładniczego lub parametry p, d, q dla modelu ARIMA.

W przypadku modeli uczenia maszynowego użytkownik wybiera zarówno typ modelu, jak i jego parametry. Przykładowo dla modeli Ridge oraz Lasso wybierany jest parametr regularyzacji α , dla modelu Random Forest liczba drzew ($n_estimators$) oraz maksymalna głębokość drzewa (max_depth), dla modelu SVR typ jądra ($kernel$) oraz parametr regularyzacji C , natomiast dla modelu XGBoost parametry takie jak liczba drzew, głębokość drzewa czy współczynnik uczenia ($learning_rate$).

Dodatkowo użytkownik może wybrać sposób doboru parametrów. Dostępne są trzy tryby konfiguracji: ustawienie ręczne, przeszukiwanie siatki (*grid search*) oraz losowe przeszukiwanie przestrzeni parametrów (*random search*).

- **Trenowanie modeli.** Po zapisaniu konfiguracji następuje etap trenowania modeli na zbiorze treningowym. W przypadku modeli uczenia maszynowego program odczytuje wybrany model, jego parametry oraz tryb ich doboru, a następnie przeprowadza uczenie modelu. Przy konfiguracji ręcznej model trenowany jest bezpośrednio z podanymi parametrami. W przypadku *grid search* wykonywane jest systematyczne przeszukiwanie wszystkich kombinacji parametrów z wcześniej zdefiniowanej siatki wartości. Natomiast w przypadku *random search* parametry są losowane z określonych zakresów, a model jest trenowany dla ograniczonej liczby losowo wybranych kombinacji, po czym zapisywany jest najlepszy uzyskany model.

W przypadku metod tradycyjnych trenowanie wykonywane jest tylko dla modeli, które tego wymagają, takich jak proste wygładzanie wykładnicze oraz ARIMA. Dla metod takich jak prognoza naiwna lub średnia ruchoma nie jest wymagane klasyczne uczenie modelu, a zapisywana jest jedynie informacja o wybranej metodzie i jej parametrach.

- **Generowanie prognoz i ocena modeli.** Po wytrenowaniu modeli generowane są prognozy dla zbioru testowego. Następnie obliczane są podstawowe miary jakości prognoz:
 - MAE – średni błąd bezwzględny,
 - RMSE – pierwiastek z średniego błędu kwadratowego,
 - MAPE – średni procentowy błąd bezwzględny,
 - R^2 – współczynnik determinacji.

Na podstawie tych miar tworzona jest tabela porównawcza umożliwiającą ocenę jakości prognoz generowanych przez modele tradycyjne oraz modele uczenia maszynowego.

- **Warianty eksperymentów.** Program umożliwia przeprowadzanie eksperymentów w różnych wariantach:
 - wariant statyczny – model jest trenowany raz na zbiorze treningowym, a na-

stępnie generuje prognozę dla całego zbioru testowego bez dalszej aktualizacji parametrów,

- wariant kroczący – prognozowanie wykonywane jest krok po kroku dla kolejnych obserwacji. Po wygenerowaniu prognozy dla danego okresu rzeczywista wartość tej obserwacji jest dodawana do zbioru treningowego, a model jest ponownie trenowany przed wygenerowaniem kolejnej prognozy.

- **Rozszerzona ocena modeli z wykorzystaniem symulacji kontroli zapasów.** Oprócz klasycznych miar jakości prognozy moduł oceny został rozszerzony o prostą symulację kontroli zapasów. W symulacji przyjęto stały początkowy poziom zapasu, poziom zapasu bezpieczeństwa oraz ustalony horyzont planowania zamówienia. W każdym kroku czasowym stan magazynowy był pomniejszany o rzeczywistą wartość popytu ze zbioru testowego. Jeżeli poziom zapasu spadał poniżej zera, rejestrowany był dzień braku zapasu. Jeżeli natomiast poziom zapasu spadał poniżej poziomu bezpieczeństwa, generowane było zamówienie uzupełniające na podstawie sumy prognozowanego popytu dla kolejnych dni.

Po uruchomieniu funkcji `evaluate_model()` wyniki są zwracane w postaci struktury danych zawierającej metryki dla obu porównywanych modeli. Oprócz miar MAE, RMSE, MAPE oraz R^2 prezentowane są również dodatkowe wskaźniki związane z symulacją magazynu, takie jak liczba dni braku zapasu (*stockout_days*) oraz średni poziom zapasu (*avg_stock*). Dzięki temu możliwa jest ocena modeli nie tylko pod względem dokładności prognozy, ale również ich wpływu na poziom zapasu oraz ryzyko wystąpienia braków magazynowych.

Kod źródłowy narzędzia opracowanego w ramach pracy został udostępniony w repozytorium GitHub: https://github.com/jarekwan/praca_inzynierska

6. Testy modeli prognozowania popytu i zarządzania zapasami

W tym rozdziale przedstawione są przypadki użycia wybranych algorytmów uczenia maszynowego – na wybrane problemy łańcucha dostaw w obszarach: prognozowania popytu oraz zarządzania zapasami.

6.1. Test prognozowania popytu

Tradycyjne metody prognozowania popytu, opierające się głównie na analizie szeregów czasowych, mają ograniczoną zdolność do przewidywania krótko- i średnioterminowych zmian, szczególnie gdy dotyczą danych charakteryzujących się zmiennością.

Metody wykorzystane w analizie W eksperymentach porównano tradycyjne metody prognozowania popytu oraz modele uczenia maszynowego.

Do metod tradycyjnych wykorzystanych w analizie należą: metoda naiwna, średnia ruchoma, proste wygładzanie wykładnicze (SES) oraz model ARIMA.

Do algorytmów uczenia maszynowego wykorzystanych w eksperymentach należą: Linear Regression, Ridge, Lasso, Random Forest, Support Vector Regression (SVR), XGBoost oraz MLP (Multilayer Perceptron).

Metody te zostały opisane we wcześniejszych rozdziałach pracy, dlatego w tej części przedstawiono jedynie wyniki ich zastosowania w zadaniu prognozowania popytu.

Kryterium oceny dokładności prognozowania popytu Mean Absolute Percentage Error (MAPE), czyli średni absolutny błąd procentowy, to miara stosowana do oceny dokładności modelu prognozowania. Wyrażona jest w procentach.

$$\text{MAPE} = \frac{1}{n} \sum_{t=1}^n \left| \frac{A_t - F_t}{A_t} \right| \times 100 \quad (19)$$

gdzie: - n : liczba obserwacji, - A_t : rzeczywista wartość w momencie t , - F_t : prognozowana wartość w momencie t .

Im niższy MAPE, tym bardziej dokładny model. Na przykład, MAPE wynoszący 10% oznacza, że prognozowane wartości średnio odbiegają o 10% od wartości rzeczywistych.

Mean Absolute Error (MAE) mierzy średnią bezwzględną różnicę pomiędzy wartościami rzeczywistymi a prognozowanymi.

$$\text{MAE} = \frac{1}{n} \sum_{t=1}^n |A_t - F_t| \quad (20)$$

gdzie:

- A_t – rzeczywista wartość w okresie t ,
- F_t – prognozowana wartość w okresie t ,
- t – indeks okresu,
- n – liczba obserwacji.

Root Mean Squared Error (RMSE) mierzy przeciętny błąd prognozy, przy czym większe błędy są silniej karane.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{t=1}^n (A_t - F_t)^2} \quad (21)$$

gdzie:

- A_t – rzeczywista wartość w okresie t ,
- F_t – prognozowana wartość w okresie t ,
- t – indeks okresu,
- n – liczba obserwacji.

Współczynnik determinacji R^2 informuje, jaka część zmienności danych rzeczywistych jest wyjaśniana przez model.

$$R^2 = 1 - \frac{\sum_{t=1}^n (A_t - F_t)^2}{\sum_{t=1}^n (A_t - \bar{A})^2} \quad (22)$$

gdzie:

- A_t – rzeczywista wartość w okresie t ,
- F_t – prognozowana wartość w okresie t ,
- $\bar{A}$ – średnia wartość rzeczywista,
- t – indeks okresu,
- n – liczba obserwacji [41].

Dane użyte do testowania (dataset) Dane użyte do testowania pochodzą z platformy Kaggle i dotyczą sprzedaży produktów marki Adidas na rynku Stanów Zjednoczonych. W analizie wykorzystano zbiór danych *Adidas US Sales Dataset*, dostępny pod adresem: <https://www.kaggle.com/datasets/sagarmorework/adidas-us-sales>.

Zbiór obejmuje codzienne rekordy sprzedaży zawierające m.in. informacje o detaliście, dacie sprzedaży, lokalizacji (region, stan, miasto), rodzaju produktu, cenie jednostkowej, liczbie sprzedanych jednostek, wartości sprzedaży, zysku, marży operacyjnej oraz kanale dystrybucji.

Dla tej samej daty występuje wiele obserwacji wynikających z podziału sprzedaży na różne produkty, lokalizacje oraz kanały sprzedaży.

Wstępna analiza danych wskazuje na brak wyraźnego trendu oraz stabilnej sezonowości, natomiast sprzedaż charakteryzuje się dużymi, nieregularnymi zmianami pomiędzy kolejnymi dniami.

Zbiór danych zawiera 9637 obserwacji oraz 13 zmiennych opisujących transakcje sprzedaży produktów marki Adidas. Dane obejmują okres od 1 stycznia 2020 roku do 31 grudnia 2021 roku [42].

W tabeli 6.1 przedstawiono próbkę danych:

Tabela 6.1. Próbkę danych – Adidas US Sales Dataset

Retailer	Retailer ID	Invoice Date	Region	State	City	Product	Price per Unit	Units Sold	Total Sales	Operating Profit	Operating Marg	Sales Method
Foot Locker	1185732	01.01.2020	Northeast	New York	New York	Men's Street Footwear	\$50,00	1 200	\$600 000	\$300 000	50%	In-store
Foot Locker	1185732	01.01.2020	Northeast	Pennsylvania	Philadelphia	Women's Apparel	\$75,00	275	\$206 250	\$61 875	30%	Outlet
Foot Locker	1185732	01.01.2020	Northeast	New York	New York	Men's Street Footwear	\$47,00	336	\$15 792	\$9 633	61%	Online
Foot Locker	1185732	01.01.2020	Northeast	Pennsylvania	Philadelphia	Women's Apparel	\$68,00	83	\$5 644	\$2 427	43%	Online
Foot Locker	1185732	01.01.2020	Northeast	New York	New York	Men's Street Footwear	\$34,00	384	\$13 056	\$6 789	52%	Outlet

W kolejnych testach dokonano modyfikacji danych poprzez zastosowanie filtrowania po rodzaju produktu. W analizie wykorzystano zarówno kategorię Women's Street Footwear, jak i Men's Street Footwear.

6. Testy modeli prognozowania popytu i zarządzania zapasami

Kolejnym zbiorem danych wykorzystanym w analizie jest *Superstore Sales Dataset 2016–2018* pochodzący z platformy Kaggle, dostępny pod adresem: <https://www.kaggle.com/datasets/rohitsahoo/sales-forecasting>.

Zbiór zawiera około 10 000 rekordów transakcji sprzedażowych z kilkuletniego okresu działalności sieci handlowej. Każdy rekord opisuje pojedynczą transakcję i obejmuje m.in. identyfikator zamówienia, datę zamówienia i wysyłki, sposób dostawy, lokalizację sprzedaży (miasto, stan, region), kategorię i nazwę produktu, liczbę sprzedanych jednostek, wartość sprzedaży, poziom rabatu oraz zysk ze sprzedaży [43].

Ze względu na brak informacji o rzeczywistym źródle danych zbiór potraktowano jako syntetyczny i wykorzystano wyłącznie do celów analitycznych oraz porównania metod prognostycznych.

W tabeli 6.2 przedstawiono próbkę danych:

Tabela 6.2. Próbkę danych – Superstore Sales Dataset

Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID
CA-2017-15215	08.11.2017	11.11.2017	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-BO-100F
CA-2017-15215	08.11.2017	11.11.2017	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-CH-100F
CA-2017-13868	12.06.2017	16.06.2017	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036	West	OFF-LA-100C
US-2016-10896	11.10.2016	18.10.2016	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-100F
US-2016-10896	11.10.2016	18.10.2016	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	OFF-ST-100C

Program testowy Program wykorzystany do przeprowadzenia eksperymentów został opisany w rozdziale 5. W niniejszym rozdziale przedstawiono jedynie sposób wykorzystania w procesie testowania modeli prognozowania oraz oceny uzyskanych wyników.

Testowanie modeli W pierwszej kolejności testy przeprowadzono na danych sprzedaży produktów marki Adidas na rynku Stanów Zjednoczonych w latach 2020–2021.

Testowanie modeli odbywało się etapowo, przy zachowaniu tej samej procedury przygotowania danych dla każdego testu. Celem testów było porównanie skuteczności różnych podejść prognostycznych w warunkach danych o wysokim poziomie szumu oraz zmienności.

W analizie uwzględniono dwa główne podejścia: modele tradycyjne szeregów czasowych oraz modele uczenia maszynowego.

Każde z podejść zostało dodatkowo przetestowane w dwóch wariantach:

wariant statyczny – model uczony na zbiorze treningowym i generujący prognozę bez aktualizacji w trakcie okresu testowego,

wariant kroczący – model, którego prognoza była aktualizowana krok po kroku z wykorzystaniem nowych obserwacji (przesuniętego okna czasowego).

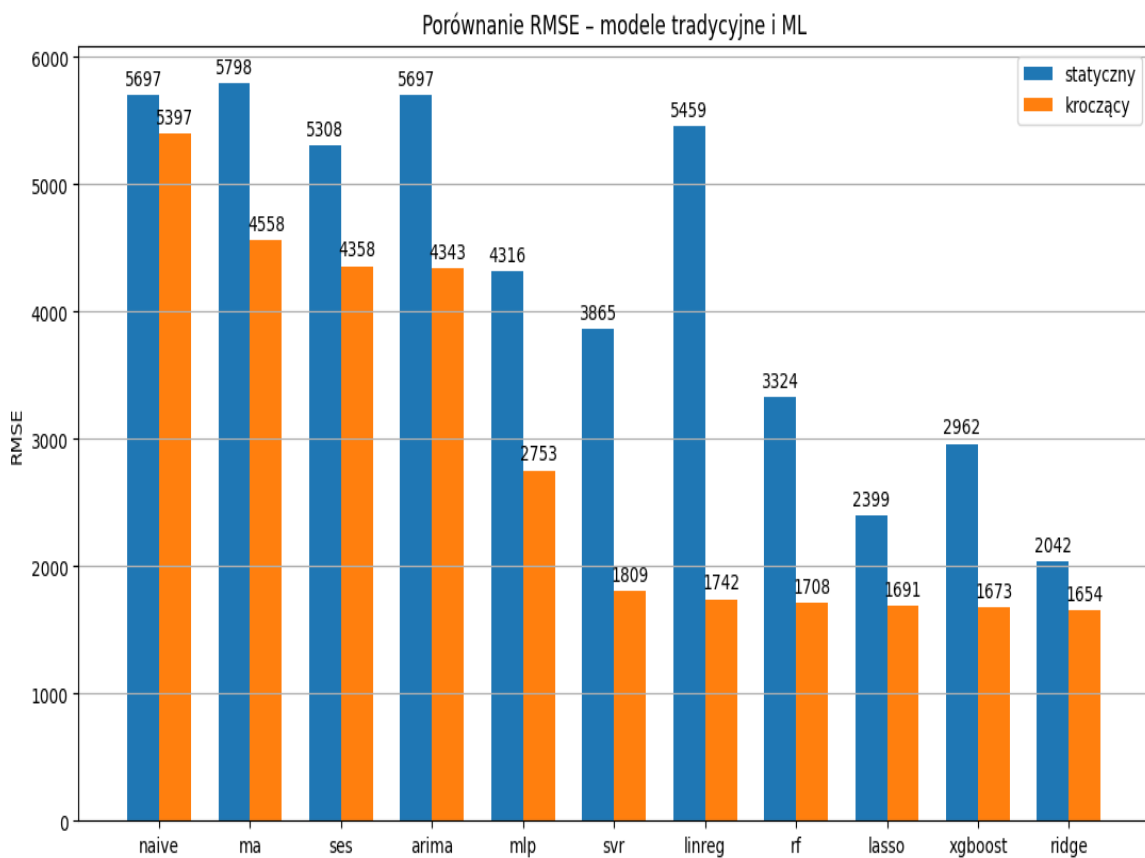
Wśród modeli tradycyjnych uwzględniono:

naiwną prognozę bazującą na ostatniej obserwacji, średnie kroczące o różnych długościach okna, wygładzanie wykładnicze (SES) z różnymi wartościami parametru wygładzania oraz modele ARIMA o kilku konfiguracjach parametrów.

W grupie modeli uczenia maszynowego zastosowano: regresję liniową oraz jej wersje (Ridge, Lasso), las losowy, maszyny wektorów nośnych, gradient boosting (XGBoost) oraz sieci neuronowe typu MLP.

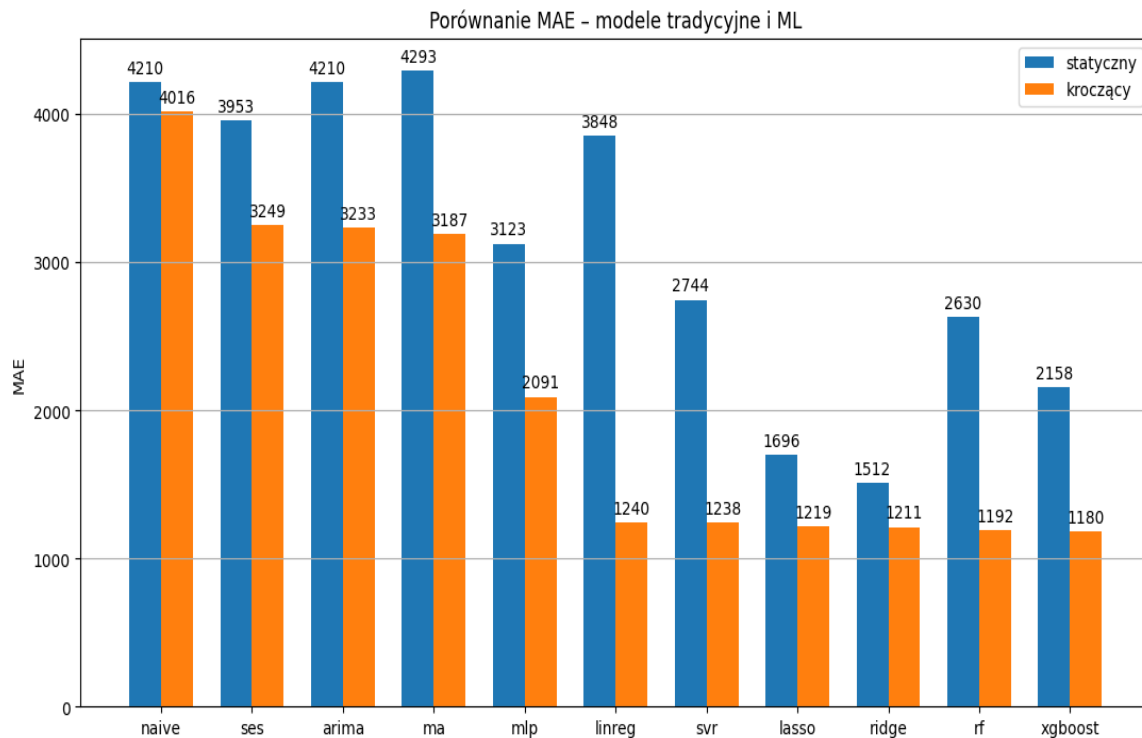
Parametry modeli były dobierane trzema metodami: ręcznie (manual), z użyciem przeszukiwania siatki (grid search) oraz losowego przeszukiwania przestrzeni parametrów (random search).

Wyniki testów:

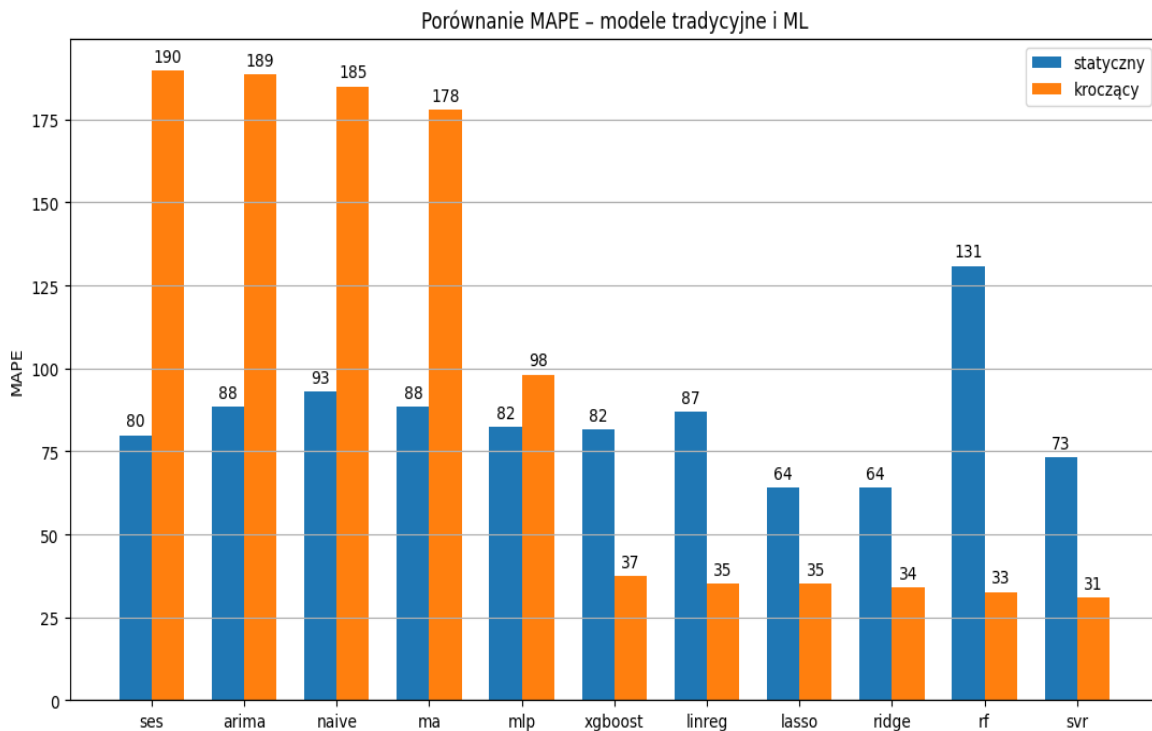


Rysunek 6.1. Porównanie wartości RMSE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym

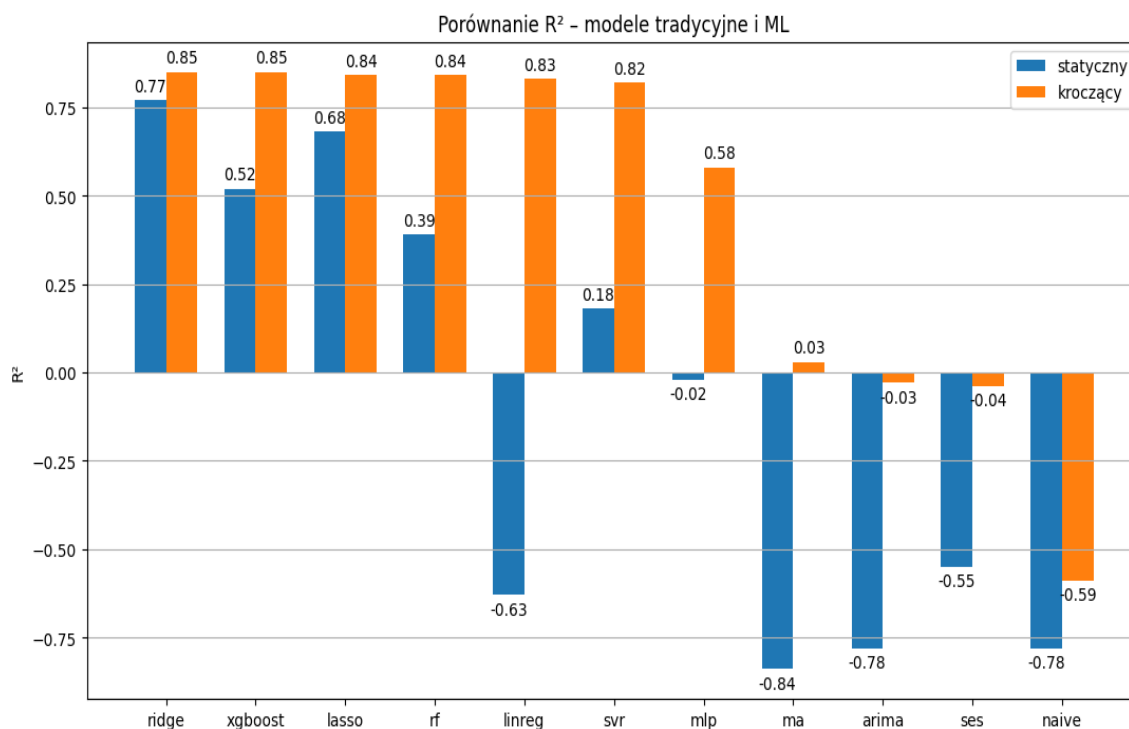
6. Testy modeli prognozowania popytu i zarządzania zapasami



Rysunek 6.2. Porównanie wartości MAE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym



Rysunek 6.3. Porównanie wartości MAPE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym



Rysunek 6.4. Porównanie wartości R^2 dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym

Analiza wyników przedstawionych na rysunkach wskazuje wyraźne różnice pomiędzy analizowanymi podejściami prognostycznymi. Modele uczenia maszynowego osiągały niższe wartości błędów prognozowania (RMSE, MAE, MAPE) oraz wyższe wartości współczynnika determinacji R^2 niż tradycyjne metody szeregów czasowych. Dodatkowo zastosowanie wariantu kroczącego prowadzi do poprawy jakości prognoz w porównaniu z wariantem statycznym.

Wyniki testów przeprowadzonych na danych charakteryzujących się wysokim poziomem szumu, niestabilnością zapotrzebowania oraz zróżnicowaniem kanałów dystrybucji i produktów wskazują, że modele tradycyjne w wariancie statycznym osiągały wysokie wartości błędów (MAE, RMSE) oraz ujemne wartości współczynnika determinacji R^2 . Zastosowanie wariantu kroczącego poprawiało wyniki tych metod, jednak poprawa ta była ograniczona i nie eliminowała problemu niestabilności danych oraz obecności szumu.

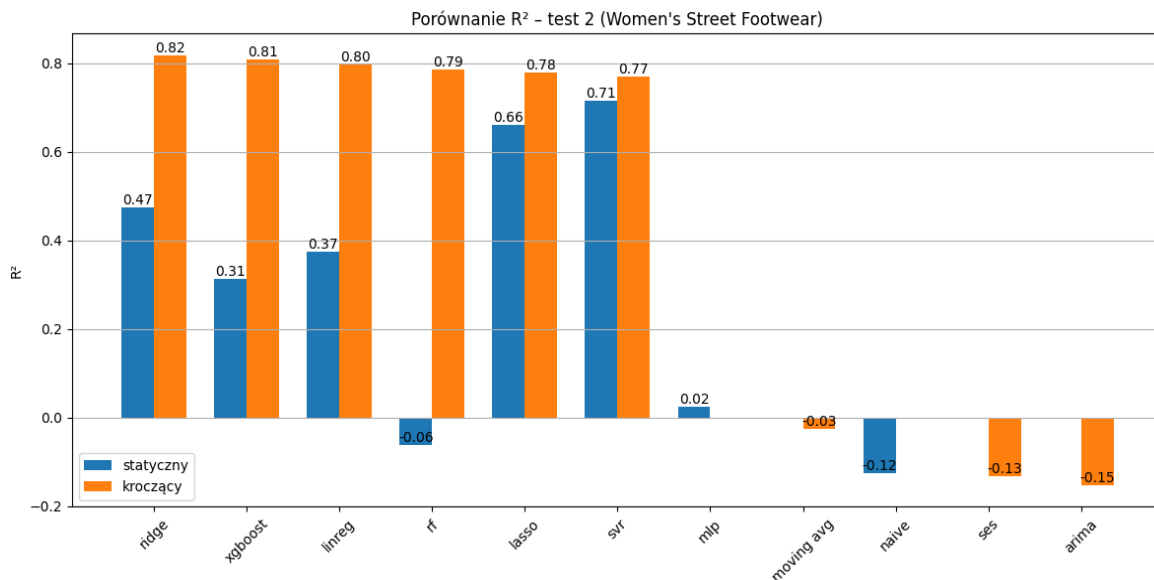
Znacząco lepsze rezultaty uzyskano dla modeli uczenia maszynowego, szczególnie w wariancie kroczącym. Modele takie jak Ridge, Lasso, Random Forest oraz XGBoost osiągały dodatnie i wysokie wartości R^2 oraz istotnie niższe błędy MAE i RMSE. Wskazuje to, że wykorzystanie cech opóźnionych oraz statystyk kroczących pozwalało modelom skuteczniej radzić sobie z niestabilnością analizowanych danych sprzedażowych.

Pełne wyniki eksperymentów dla poszczególnych konfiguracji modeli przedstawiono w tabelach Załącznik 1.1–1.4. Zawierają one surowe dane uzyskane w trakcie testów i stanowią podstawę do opracowania wykresów porównawczych.

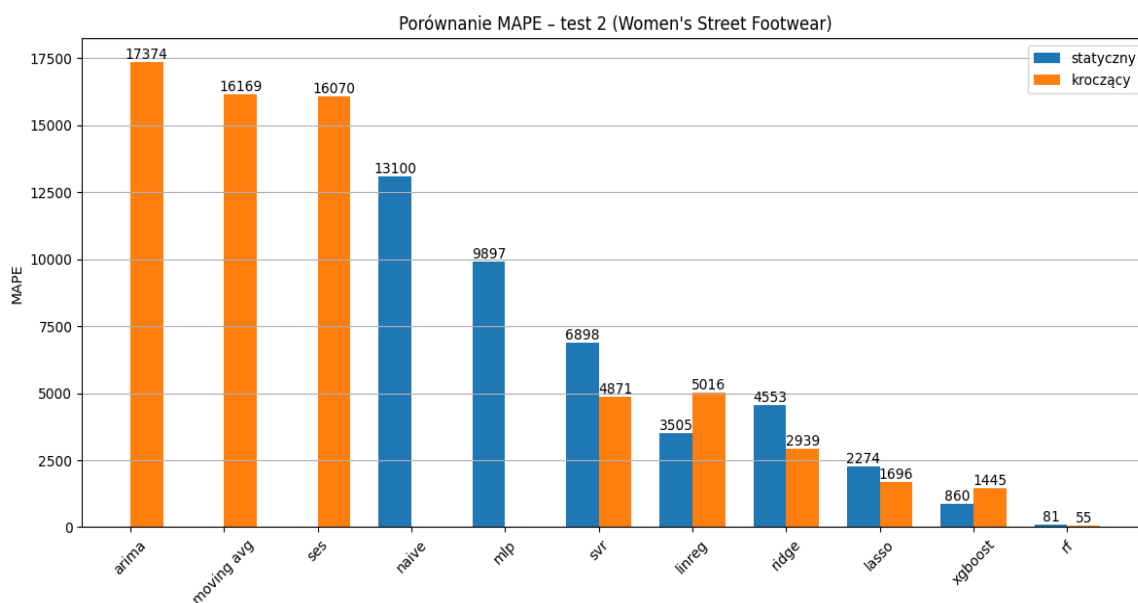
6. Testy modeli prognozowania popytu i zarządzania zapasami

Kolejny test wykonano na tych samych danych, ale po filtrowaniu po produkcie: Women's Street Footwear

Wyniki testu:

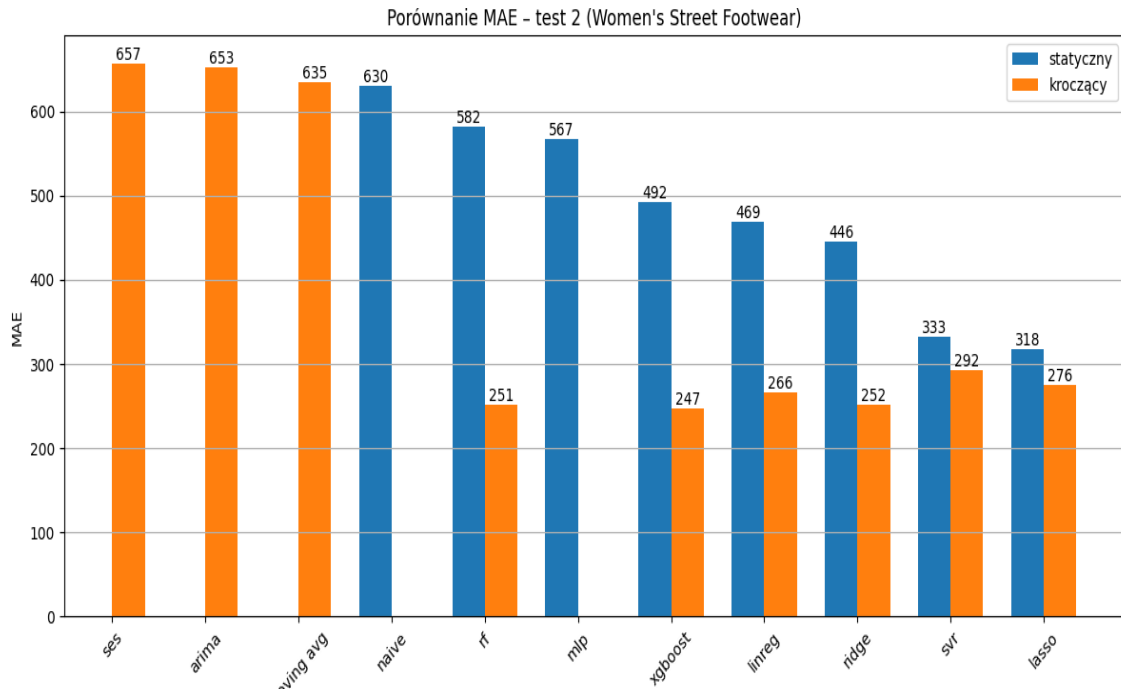


Rysunek 6.5. Porównanie wartości R^2 dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym

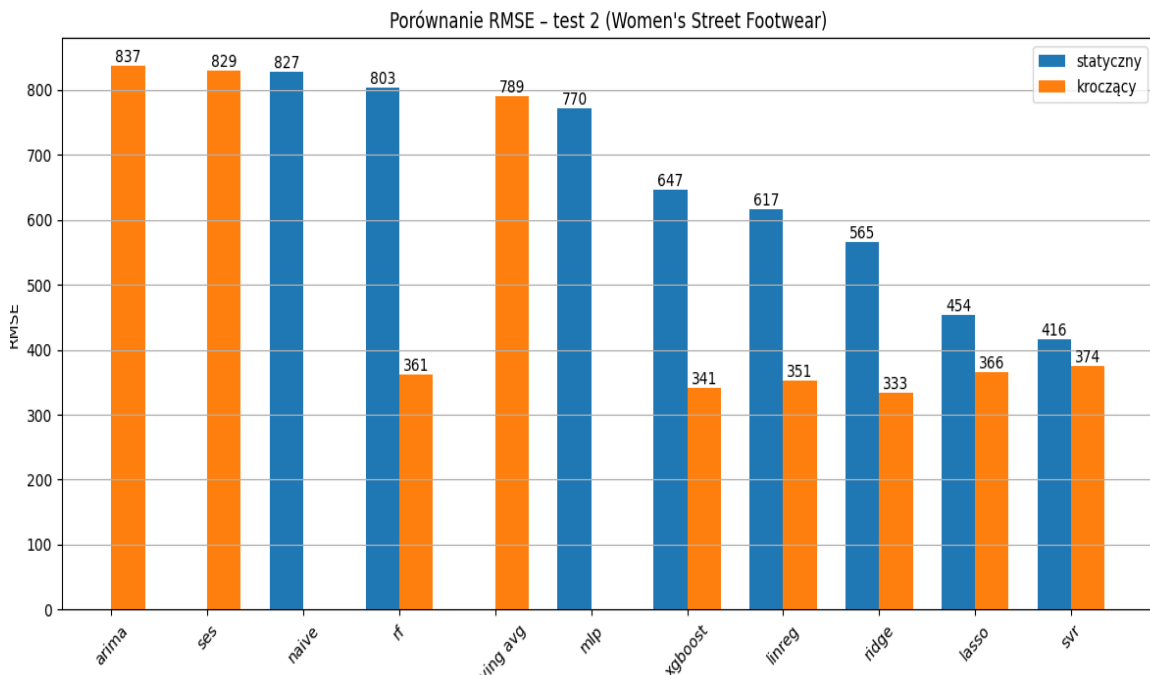


Rysunek 6.6. Porównanie wartości MAPE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym

6. Testy modeli prognozowania popytu i zarządzania zapasami



Rysunek 6.7. Porównanie wartości MAE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym



Rysunek 6.8. Porównanie wartości RMSE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym

Analiza wyników przedstawionych na rysunkach wskazuje, że również w przypadku danych przefiltrowanych do produktu Women's Street Footwear modele uczenia maszynowego osiągają lepsze wyniki niż tradycyjne metody szeregów czasowych. Widoczne

jest obniżenie wartości błędów prognozowania (RMSE, MAE, MAPE) oraz wzrost wartości współczynnika determinacji R^2 , szczególnie dla modeli Ridge, Lasso, SVR oraz XGBoost. Podobnie jak w poprzednim teście, zastosowanie wariantu kroczącego prowadzi do dalszej poprawy jakości prognoz w porównaniu z wariantem statycznym.

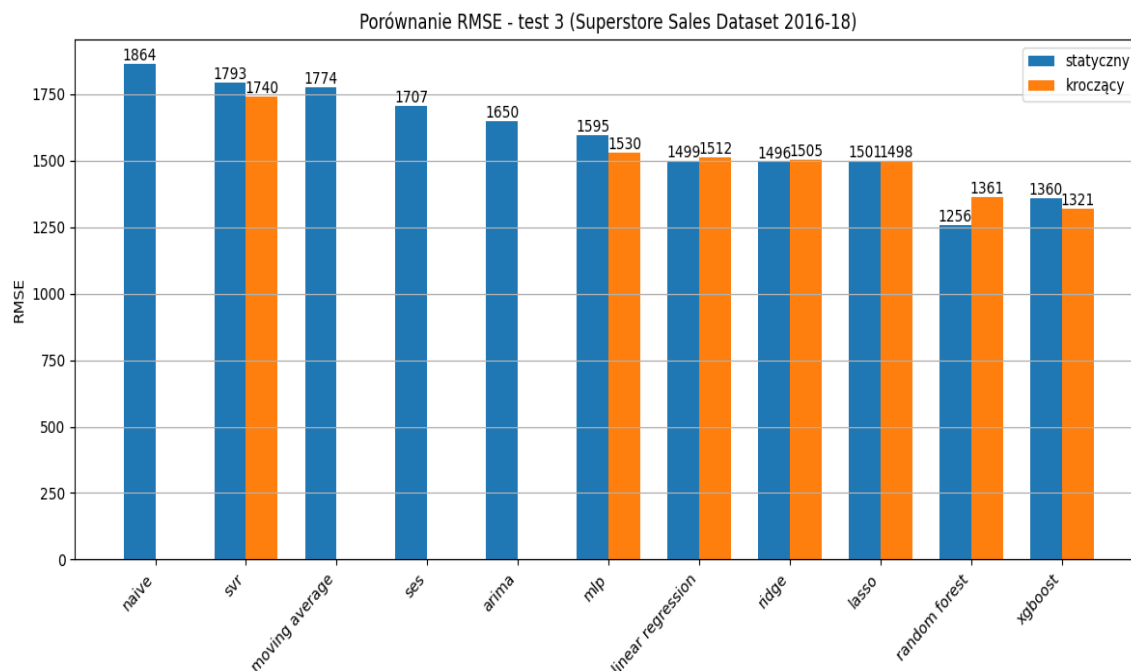
W drugim teście wykorzystano dane po filtracji do jednej grupy produktowej, co ograniczyło poziom szumu i niestabilność danych sprzedażowych. Dzięki temu jakość prognoz wyraźnie się poprawiła zarówno dla metod statycznych, jak i kroczących. Modele statyczne uczenia maszynowego osiągnęły zdecydowanie lepsze wyniki niż w teście pierwszym, szczególnie w przypadku regresji z regularyzacją. Najlepsze rezultaty w tej grupie uzyskały modele Lasso ($\alpha = 100$) oraz SVR z jądrem liniowym ($C = 100$).

Modele kroczące uczenia maszynowego również poprawiły swoje wyniki. Szczególnie dobre rezultaty uzyskały Ridge Regression ($\alpha = 10$) oraz XGBoost ($n = 200$, $depth = 4$), osiągając wysokie wartości współczynnika R^2 przy jednocześnie niskich błędach MAE i RMSE. Tradycyjne metody prognozowania uzyskały jedynie niewielką poprawę wyników, co wskazuje, że nawet filtracja danych nie pozwoliła im osiągnąć skuteczności porównywalnej z modelami uczenia maszynowego.

Pełne wyniki eksperymentów dla poszczególnych konfiguracji modeli przedstawiono w tabeli zamieszczonej w Załączniku 2. Zawiera ona surowe dane uzyskane w trakcie testów.

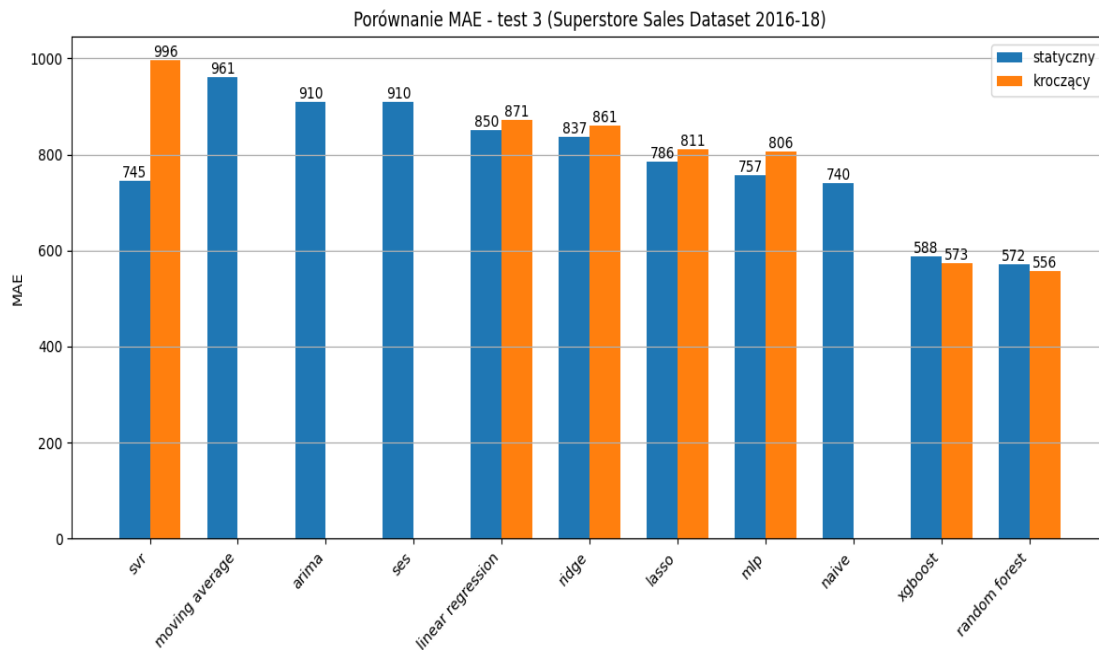
Trzeci test przeprowadzono na danych „Superstore Sales Dataset 2016–2018”.

Wyniki testu:

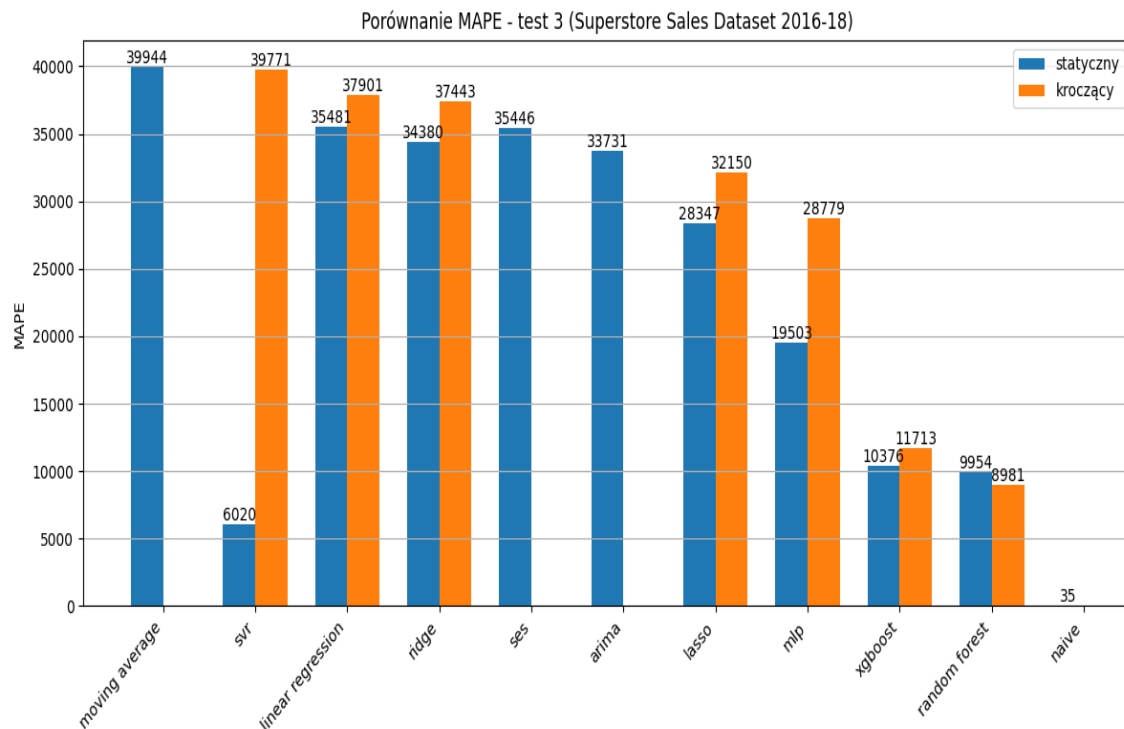


Rysunek 6.9. Porównanie wartości RMSE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariantach statycznym i kroczącym

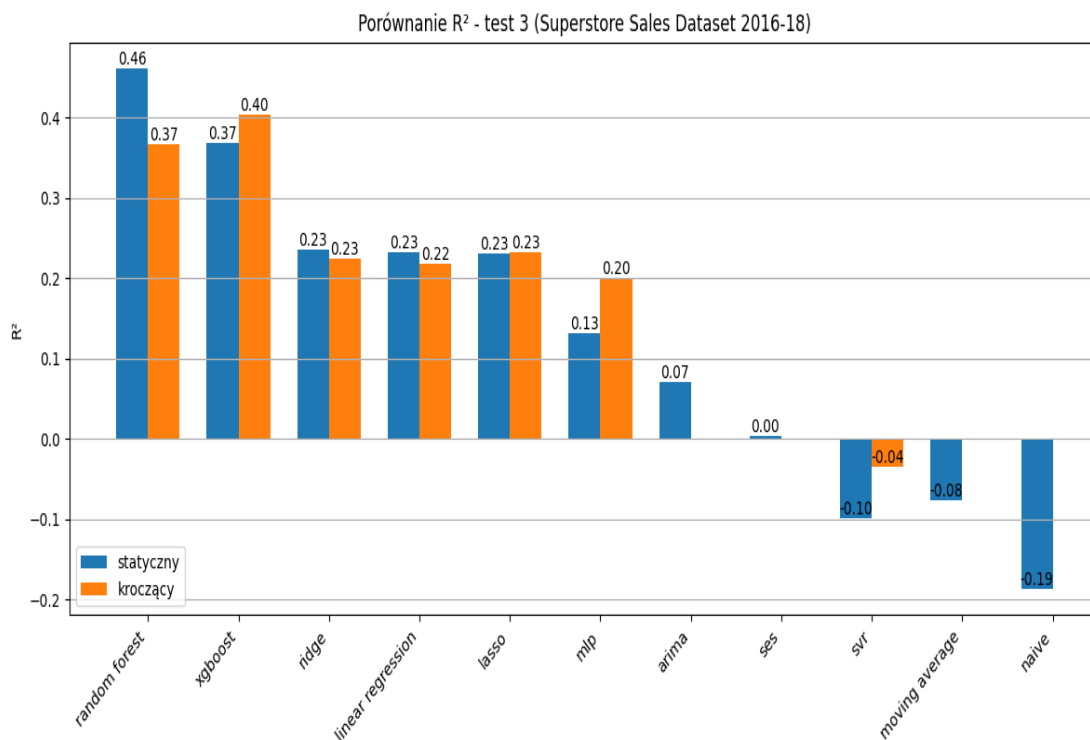
6. Testy modeli prognozowania popytu i zarządzania zapasami



Rysunek 6.10. Porównanie wartości MAE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym



Rysunek 6.11. Porównanie wartości MAPE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym



Rysunek 6.12. Porównanie wartości R^2 dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym

Analiza wyników przedstawionych na rysunkach wskazuje, że w przypadku danych z zestawu Superstore Sales Dataset 2016–18 najlepsze rezultaty uzyskały modele uczenia maszynowego. Szczególnie dobre wyniki osiągnęły modele zespołowe Random Forest oraz XGBoost, które charakteryzowały się najniższymi wartościami błędów prognozowania (MAE, RMSE) oraz najwyższymi wartościami współczynnika determinacji R^2 .

Wśród modeli statycznych najslabsze rezultaty uzyskały najprostsze metody prognozowania, takie jak prognoza naiwna, średnia krocząca oraz proste wygładzanie wykładnicze. Metody te generowały wysokie wartości błędów prognozowania, a współczynnik R^2 był często ujemny lub bliski zeru, co wskazuje na ich ograniczoną zdolność do odwzorowania zmienności danych. Lepsze wyniki osiągnęły modele regresji liniowej z regularyzacją – Ridge oraz Lasso – które uzyskały dodatnie wartości R^2 , jednak poziom błędów prognozowania nadal pozostawał stosunkowo wysoki.

Modele SVR oraz MLP w trybie statycznym osiągały niestabilne rezultaty i w wielu konfiguracjach generowały wysokie błędy oraz ujemne wartości R^2 . Zastosowanie wariantu kroczącego w większości przypadków prowadziło do poprawy jakości prognoz, jednak najlepsze wyniki w całym teście ponownie uzyskały modele zespołowe Random Forest oraz XGBoost, które osiągnęły najniższe błędy prognozy oraz najwyższe wartości współczynnika R^2 .

Pełne wyniki eksperymentów dla poszczególnych konfiguracji modeli przedstawiono

w tabeli zamieszczonej w Załączniku 3 (Tabela 3.1). Zawiera ona surowe dane uzyskane w trakcie testów.

6.2. Test zarządzania zapasami

W części dotyczącej kontroli zapasów wykorzystano filtrowane dane sprzedażowe Adidas – kategoria Men Footwear. Dodatkowo użyto zbioru Superstore Sales Dataset 2016–2018. Moduł *evaluate* został rozszerzony o prostą symulację kontroli zapasów, aby umożliwić ocenę wpływu jakości prognozy na stan magazynowy, a nie wyłącznie porównanie błędów prognostycznych.

W zmodyfikowanej wersji zachowano pełną dotychczasową funkcjonalność, a dodatkowo wprowadzono prosty model magazynowy oparty na danych testowych oraz wygenerowanych prognozach.

Symulacja kontroli zapasów prowadzona jest w krokach dziennych. Przyjęto następujące założenia:

- stały początkowy poziom zapasu (*initial_stock*),
- minimalny dopuszczalny poziom zapasu (*safety_stock*),
- horyzont planowania zamówienia równy 5 dni (*horizon_days*).

W każdym kroku czasowym realizowane są następujące operacje:

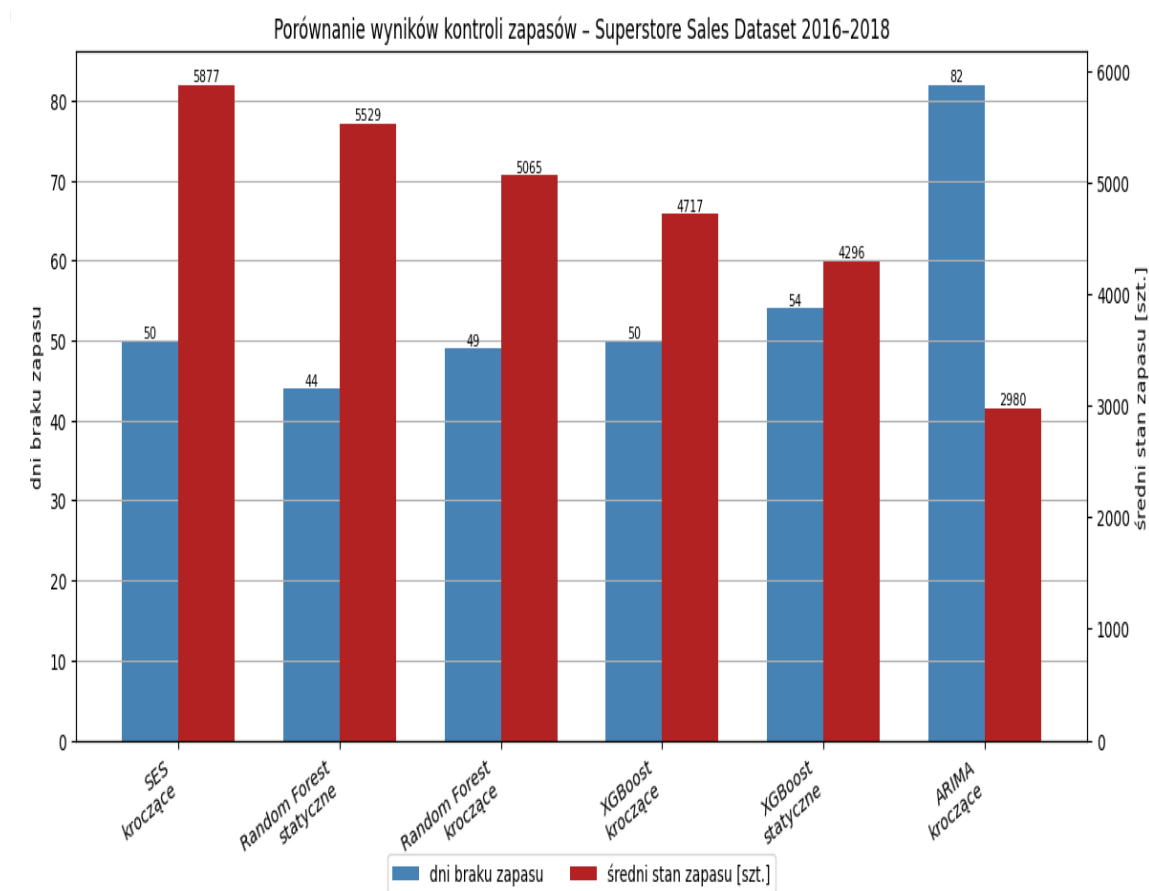
- stan magazynu jest pomniejszany o rzeczywistą sprzedaż, a nie o wartość prognozowaną,
- w przypadku spadku zapasu poniżej zera rejestrowany jest dzień braku zapasu (*stockout_days*), a stan magazynu jest zerowany,
- jeżeli poziom zapasu spadnie poniżej poziomu bezpieczeństwa, generowane jest zamówienie uzupełniające,
- wielkość zamówienia obliczana jest jako suma prognozowanego popytu na kolejne 5 dni pomniejszona o aktualny stan magazynu,
- zamówienie jest natychmiast dodawane do stanu magazynu, bez modelowania czasu dostawy,
- zapisywana jest historia poziomu zapasu w celu obliczenia średniego stanu magazynowego.

Takie podejście pozwala jednoznacznie ocenić, czy poprawa jakości prognozy przekłada się na mniejszą liczbę braków zapasu oraz niższy średni poziom zapasu.

Oprócz klasycznych miar błędu prognozy dla każdego modelu raportowane są również dodatkowe wskaźniki:

- liczba dni braku zapasu (*stockout_days*),
- średni poziom zapasu (*avg_stock*).

Testowanie modeli Wyniki 1go testu:

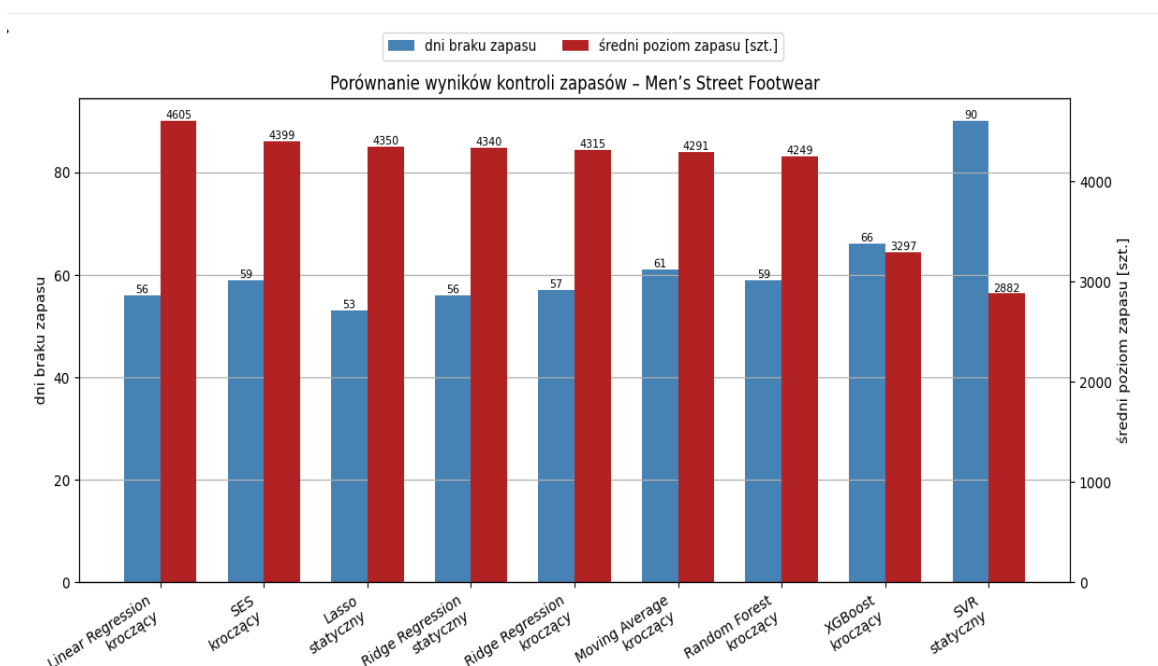


Rysunek 6.13. Porównanie wyników kontroli zapasów dla różnych modeli prognozowania – liczba dni braku zapasu oraz średni poziom zapasu

Wyniki przedstawione na rysunku wskazują, że modele oparte na uczeniu maszynowym osiągają lepsze rezultaty w kontroli zapasów niż klasyczne metody szeregów czasowych. Najlepsze wyniki uzyskały modele Random Forest oraz XGBoost. Random Forest w trybie statycznym charakteryzował się najniższą liczbą dni braku zapasu (44), jednak kosztem najwyższego średniego poziomu zapasu, natomiast XGBoost w wariantcie kroczącym osiągnął korzystniejszy kompromis pomiędzy stabilnością dostępności produktów a poziomem zapasów.

Modele klasyczne, takie jak ARIMA oraz SES, uzyskały słabsze rezultaty. Model ARIMA generował największą liczbę dni braku zapasu, natomiast metoda wykładniczego (SES) utrzymywała wysoki poziom zapasu bez wyraźnej redukcji braków.

Kolejny test i wyniki:



Rysunek 6.14. Porównanie wyników kontroli zapasów dla różnych modeli prognozowania – liczba dni braku zapasu oraz średni poziom zapasu (Men's Street Footwear)

Najniższą liczbę dni z brakiem zapasu uzyskał model Lasso w konfiguracji statycznej z wyszukiwaniem parametrów metodą grid search, osiągając 53 dni braku zapasu (stockoutu).

Modele Ridge Regression, zarówno w trybie statycznym, jak i kroczącym, oraz Linear Regression w trybie kroczącym osiągnęły zbliżone wyniki w zakresie 56–57 dni braku zapasu przy umiarkowanym poziomie zapasów.

Model XGBoost w trybie kroczącym (grid search) miał wyraźnie niższy średni poziom zapasu (3297), jednak kosztem większej liczby dni z brakiem zapasu (66).

Najgorsze wyniki uzyskał model SVR w konfiguracji statycznej, dla którego liczba dni braku zapasu była najwyższa (90).

Podsumowanie Na podstawie przeprowadzonych testów kontroli zapasów można stwierdzić, że żaden z analizowanych modeli nie dominuje w obu kryteriach jednocześnie. Najlepszy kompromis między liczbą dni braku zapasu a średnim stanem magazynowym osiągnęły modele zespołowe – Random Forest oraz XGBoost – które przy relatywnie wysokim poziomie zapasu skutecznie ograniczały ryzyko niedoborów. Klasyczne metody szeregów czasowych, takie jak ARIMA i SES, utrzymywały niższe stany magazynowe, jednak kosztem znacznie większej liczby dni z brakiem towaru, co w praktyce dyskwalifikuje je jako samodzielne narzędzie sterowania zapasami. Tryb prognozowania – statyczny bądź kroczący – okazał się czynnikiem drugorzędym w stosunku do wyboru klasy modelu,

co sugeruje, że to architektura algorytmu, a nie częstotliwość aktualizacji parametrów, w największym stopniu determinuje jakość polityki zapasów.

Pełne wyniki eksperymentów przedstawiono w tabelach zamieszczonych w Załączniku 4 (Tabele 4.1–4.2). Zawierają one surowe wartości uzyskane w symulacji kontroli zapasów.

7. Wyniki i podsumowanie

Podsumowanie wyników analiz Przeprowadzone eksperymenty miały na celu porównanie skuteczności klasycznych metod prognozowania popytu z algorytmami uczenia maszynowego w zarządzaniu łańcuchem dostaw.

Testy wykonano na trzech zbiorach danych charakteryzujących się różnym poziomem szumu oraz stopniem agregacji.

Najważniejsze wnioski można sformułować następująco:

- W warunkach wysokiego szumu i nieregularnej sprzedaży metody klasyczne osiągały znacząco wyższe błędy prognozy (mierzone m.in. MAPE, MAE, RMSE) oraz niskie, często ujemne wartości współczynnika determinacji R^2 .
- Modele uczenia maszynowego (w szczególności Random Forest, XGBoost oraz regresje z regularyzacją – Ridge i Lasso) uzyskiwały wyraźnie lepszą jakość prognoz (niższy błąd) w porównaniu z najlepszymi wariantami metod statystycznych.
- Wariant uczenia kroczącego (rolling window) przynosił dodatkową poprawę dokładności w większości przypadków dzięki uwzględnieniu najnowszych obserwacji.
- Redukcja szumu (ograniczenie analizy do jednej kategorii produktowej lub bardziej homogenicznych grup) poprawiała wyniki niemal wszystkich modeli, ale największy względny zysk osiągały algorytmy zespołowe i regularyzowane.
- Najbardziej stabilne i uniwersalne zachowanie wykazały modele zespołowe (Random Forest, XGBoost) oraz regresje liniowe z regularyzacją (Ridge, Lasso).

Z punktu widzenia praktycznego zarządzania łańcuchem dostaw wyższa dokładność prognoz przekłada się na:

- zmniejszenie ryzyka utraty sprzedaży z powodu braku towaru,
- redukcję nadmiernych zapasów i związanych z nimi kosztów magazynowania oraz kapitału zamrożonego,
- poprawę poziomu obsługi klienta (dostępność produktu, OTIF, fill-rate),
- stabilniejsze planowanie produkcji oraz zamówień do dostawców (zmniejszenie efektu bullwhip).

Należy jednak podkreślić istotną rolę jakości danych – nieadekwatne czyszczenie, błędna inżynieria cech lub obecność silnie skorelowanych / nieistotnych zmiennych mogą prowadzić do sytuacji, w której nawet zaawansowane modele ML osiągają wyniki gorsze niż proste metody statystyczne.

Zastosowanie algorytmów uczenia maszynowego w prognozowaniu popytu stanowi więc wartościowe, choć niepozbawione pułapek, narzędzie wspierające decyzje operacyjne w łańcuchu dostaw.

Kierunki dalszych analiz i perspektywy praktycznego wdrożenia Przeprowadzone analizy stanowią jedynie wycinek szerszego problemu integracji uczenia maszynowego z procesami planowania łańcucha dostaw. Istnieje kilka naturalnych kierunków dalszego rozwoju tematu, które mogą być przedmiotem prac magisterskich, doktorskich lub projektów wdrożeniowych w najbliższych latach:

- Integracja multimodalnych i heterogenicznych źródeł danych – w szczególności danych zewnętrznych (ceny surowców, wskaźniki makroekonomiczne, sentyment mediów społecznościowych, dane pogodowe) oraz danych strumieniowych w czasie rzeczywistym (IoT, czyli internet rzeczy, sprzedaż POS). Pozwoli to na przejście od klasycznego prognozowania do tzw. *demand sensing* (czyli szybkiego wykrywania zmian popytu).
- Rozwój modeli hybrydowych łączących głębokie uczenie (np. LSTM) z metodami statystycznymi oraz algorytmami drzewiastymi (XGBoost).
- Wdrożenie technik XAI (explainable AI) w celu zwiększenia zaufania biznesowego do „czarnych skrzynek” ML oraz ułatwienia akceptacji przez planistów i menedżerów.

W perspektywie najbliższych lat najbardziej prawdopodobne zmiany w rzeczywistych łańcuchach dostaw to:

- upowszechnienie się agentycznego AI (agentic AI) – autonomicznych agentów podejmujących lub rekomendujących decyzje na podstawie prognoz w czasie rzeczywistym,
- integracja prognoz ML bezpośrednio z systemami ERP / APS / WMS poprzez API ,

Podsumowując, uczenie maszynowe przestało być jedynie obiecującą koncepcją – staje się standardowym narzędziem operacyjnym w nowoczesnych łańcuchach dostaw. Kluczowym wyzwaniem w najbliższych latach nie będzie już samo uzyskanie niższego błędu prognozy, lecz skuteczne wdrażanie, utrzymanie oraz ciągłe doskonalenie tych modeli w środowisku biznesowym.

Bibliografia

- [1] Kaggle, *Kaggle*, Dostęp zdalny (20.01.2026): <https://www.kaggle.com/>, 2026.
- [2] T. Małkusa i S. Wiśniowska, *Zarządzanie łańcuchem dostaw*, Dostęp zdalny (20.01.2026): https://mfiles.pl/pl/index.php/łańcuch_dostaw, 2023.
- [3] W. przewodnik logistyczny, *Co to jest łańcuch dostaw? Przewodnik*, Dostęp zdalny (20.01.2026): <https://wdx.pl/2021/10/28/co-to-jest-lancuch-dostaw-przewodnik/>, 2021.
- [4] S. Osowski i R. Szmurło, *Matematyczne modele uczenia maszynowego w językach MATLAB i PYTHON*. Oficyna Wydawnicza Politechniki Warszawskiej, 2023.
- [5] S. Skoczypiec, *Metody prognozowania: Szeregi czasowe*, Dostęp zdalny (20.01.2026): <https://m6.pk.edu.pl/>, 2009.
- [6] ekonometria.4me.pl, *Metoda Wintersa addytywna i multiplikatywna*, Dostęp zdalny (20.01.2026): <http://www.ekonometria.4me.pl/metoda-wintersa.htm>, 2023.
- [7] K. Kolasińska-Morawska, *Zarządzanie logistyczne*, Dostęp zdalny (20.01.2026): <https://share.google/Ndt7qIaeA6d8RCQyN>, 2021.
- [8] D. Kryszpin i A. Flig, *Zarządzanie zapasami*, Dostęp zdalny (20.01.2026): https://mfiles.pl/pl/index.php/Zarzadzanie_zapasami, 2023.
- [9] S. Govindaiah i M. D. Petty, "Applying reinforcement learning to plan manufacturing material handling", *Discover Artificial Intelligence (2021)* 1:8, 2021.
- [10] T. Ryback, L. Lingitz, A. G. V. Gallina i D. Gyulai, "Improving the Planning Quality in Production Planning and Control with Machine Learning", 16th IMEKO TC10 Conference "Testing, Diagnostics Inspection as a comprehensive value chain for Quality Safety Berlin, Germany, on September 3-4, 2019, 2019.
- [11] G. Aurelien, *uczenie maszynowe z uzyciem scikit-learn i tensorflow*. Helion, 2020.
- [12] S. Miśtak, *Podział modeli uczenia maszynowego wraz z przykładami zastosowania*, Dostęp zdalny (20.01.2026): <https://www.gov.pl/web/popcwsparcie/podzial-modeli-uczenia-maszynowego-wraz-z-przykladami-zastosowania>, 2023.
- [13] M. Mamczur, *Czym jest uczenie maszynowe? I jakie są jego rodzaje?*, Dostęp zdalny (20.01.2026): <https://miroslawmamczur.pl/czym-jest-uczenie-maszynowe-i-jakie-sa-rodzaje/>, 2019.
- [14] K. G. Nowadly i S. Jung, "Using Machine Learning Approaches to Improve Long-Range Demand Forecasting", prac. mag., MASSACHUSETTS INSTITUTE OF TECHNOLOGY, 2020.
- [15] R. Carbonneau, K. Laframboise i R. Vahidov, "Application of machine learning techniques for supply chain demand forecasting", *European Journal of Operational Research* 184 (2008) 1140–1154, 2006.
- [16] J. Shahrabi, S. Mousavi i M. Heydar, "Supply Chain Demand Forecasting; A Comparison of Machine Learning Techniques and Traditional Methods", *Journal of Applied Sciences* 9 (3): 521 -527, 2009, 2009.

- [17] R. Bergsma, C. de Ruijter i S. Bhulai, "A systematic review of machine learning approaches in inventory control optimization", *Operation research perspectives* · 15 (2025), 2025.
- [18] J. Li, H. Huang i Y. Gong, "A Review of Artificial Intelligence in Inventory Management: Methods, Applications and Directions", *International Journal of Advanced Computer Science and Applications Vol. 16, No. 10*, 2025, 2025.
- [19] A. Sethi, *Support Vector Regression Tutorial for Machine Learning*, Dostęp zdalny (20.01.2026): <https://www.analyticsvidhya.com/blog/2020/03/support-vector-regression-tutorial-for-machine-learning/>, 2023.
- [20] G. Hansa, *What is Q-Learning? Beginners Guide*, Dostęp zdalny (20.01.2026): <https://intellipaat.com/blog/q-learning/>, 2023.
- [21] H. de Ponteves, *Sztuczna inteligencja : błyskawiczne wprowadzenie do uczenia maszynowego, uczenia ze wzmocnieniem i uczenia głębokiego*. Helion, 2021.
- [22] openai, *Welcome to Spinning Up in Deep RL*, Dostęp zdalny (20.01.2026): <https://spinningup.openai.com/en/latest/index.html>, 2023.
- [23] A. Rawat, R. Verma i R. S. Suryavanshi, "Advanced Applications of Neural Networks: From Image Recognition to Natural Language Processing", *Computational Techniques in Engineering, ISBN: 978-1-80433-922-0 Science and Technology, Edn. 2024*, 2024.
- [24] M. McGregor, *8 Clustering Algorithms in Machine Learning that All Data Scientists Should Know*, Dostęp zdalny (20.01.2026): <https://machinelearningmastery.com/clustering-algorithms-with-python/>, 2020.
- [25] Geeksforgeeks, *Support Vector Regression (SVR) using Linear and Non-Linear Kernels in Scikit Learn*, Dostęp zdalny (20.01.2026): <https://www.geeksforgeeks.org/machine-learning/support-vector-regression-svr-using-linear-and-non-linear-kernels-in-scikit-learn/>, 2023.
- [26] G. Dudek, *UCZENIE MASZYNOWE DRZEWA DECYZYJNE - REGRESJA*, Dostęp zdalny (20.01.2026): https://gdudek.el.pcz.pl/images/Dydaktyka/Wyklad4_UM_DDreg.pdf, 2023.
- [27] I. Abba, *KNN Algorithm – K-Nearest Neighbors Classifiers and Model Example*, Dostęp zdalny (20.01.2026): <https://www.freecodecamp.org/news/k-nearest-neighbors-algorithm-classifiers-and-model-example/>, 2023.
- [28] G. Zhang, *What is the kernel trick? Why is it important?*, Dostęp zdalny (20.01.2026): <https://medium.com/@zxr.nju/what-is-the-kernel-trick-why-is-it-important-98a98db0961d>, 2018.
- [29] Polsko-Japońska Akademia Technik Komputerowych, *Najpopularniejsze języki programowania – którego z nich warto się nauczyć*, Dostęp zdalny (20.01.2026): <https://pja.edu.pl/najpopularniejsze-jezyki-programowania-ktorego-z-nich-warto-sie-nauczyc/>, 2025.

-
- [30] Geeksforgeeks, *Best Language for Machine Learning*, Dostęp zdalny (20.01.2026): <https://www.geeksforgeeks.org/machine-learning/best-language-for-machine-learning/>, 2024.
- [31] Vadym Zhernovyi, *Top Programming Languages for Machine Learning*, Dostęp zdalny (20.01.2026): <https://blackthorn-vision.com/blog/top-programming-languages-for-machine-learning/>, 2024.
- [32] DistantJob, *Best Language for Machine Learning*, Dostęp zdalny (20.01.2026): <https://distantjob.com/blog/best-language-machine-learning/>, 2024.
- [33] Kaggle, *Kaggle Machine Learning and Data Science Survey 2018*, Dostęp zdalny (20.01.2026): <https://www.kaggle.com/datasets/kaggle/kaggle-survey-2018>, 2018.
- [34] Business Over Broadway, *Programming Languages Most Used and Recommended by Data Scientists*, Dostęp zdalny (20.01.2026): <https://businessoverbroadway.com/2019/01/13/programming-languages-most-used-and-recommended-by-data-scientists/>, 2019.
- [35] C3 AI, *Programming Languages for Machine Learning*, Dostęp zdalny (20.01.2026): <https://c3.ai/introduction-what-is-machine-learning/programming-languages-for-machine-learning/>, 2024.
- [36] GeeksforGeeks, *Top Programming Languages and Their Libraries for Machine Learning*, Dostęp zdalny (20.01.2026): <https://www.geeksforgeeks.org/blogs/top-programming-languages-and-their-libraries-for-machine-learning/>, 2024.
- [37] New Horizons, *Why is Python Used for Machine Learning*, Dostęp zdalny (20.01.2026): <https://www.newhorizons.com/resources/blog/why-is-python-used-for-machine-learning>, 2024.
- [38] EngineersGarage, *Machine Learning Programming Languages*, Dostęp zdalny (20.01.2026): <https://www.engineersgarage.com/machine-learning-programming-languages/>, 2022.
- [39] Medium, *What is Google Colab and How to Use it for Data Science and Machine Learning*, Dostęp zdalny (20.01.2026): <https://medium.com/@aleksej.gudkov/what-is-google-colab-and-how-to-use-it-for-data-science-and-machine-learning-20aaa61afc2e>, 2023.
- [40] Medium, *Dive into Machine Learning with Python and Google Colab*, Dostęp zdalny (20.01.2026): <https://medium.com/@baakchsu.sprx77/dive-into-machine-learning-with-python-and-google-colab-3eba4dbf6b4d>, 2023.
- [41] A. Agarwal i A. Jayand, "Support Vector Machine Model for Demand Forecasting in an Automobile parts industry: A Case Study", *Research Journal of Applied Sciences, Engineering and Technology* · July 2019, 2019.
- [42] S. More, *Adidas US Sales*, Dostęp zdalny (20.01.2026): <https://www.kaggle.com/datasets/sagarmorework/adidas-us-sales/data>, 2023.

7. Bibliografia

- [43] R. Sahoo, *Superstore Sales Dataset*, Dostęp zdalny (20.01.2026): <https://www.kaggle.com/datasets/rohitsahoo/sales-forecasting>, 2023.

Wykaz symboli i skrótów

80/20 – zasada Pareto – reguła wskazująca, że około 80% efektów wynika z około 20% przyczyn

ABC – analiza ABC – metoda klasyfikacji zapasów oparta na zasadzie Pareto, dzieląca produkty na grupy A, B i C według ich znaczenia ekonomicznego

AGENTICAI – Agentic AI – systemy sztucznej inteligencji działające w formie autonomicznych agentów podejmujących lub rekomendujących decyzje

API – Application Programming Interface – interfejs umożliwiający komunikację pomiędzy różnymi aplikacjami lub systemami

APS – Advanced Planning System – system zaawansowanego planowania w łańcuchu dostaw

ARIMA – AutoRegressive Integrated Moving Average – model szeregów czasowych łączący składnik autoregresyjny, różnicowanie oraz średnią ruchomą

BULLWHIP – Bullwhip Effect – zjawisko w łańcuchu dostaw polegające na wzmacnianiu wahań zamówień w kolejnych ogniwach łańcucha

COLAB – Google Colab – środowisko programistyczne online umożliwiające uruchamianie kodu Python w przeglądarce

CPP – C++ – język programowania ogólnego przeznaczenia wykorzystywany m.in. w systemach o wysokiej wydajności

CSV – Comma-Separated Values – format pliku tekstowego do zapisu danych tabelarycznych

DATAFRAME – DataFrame – dwuwymiarowa struktura danych w postaci tabeli z wierszami i kolumnami, wykorzystywana m.in. w bibliotece Pandas

DBSCAN – Density-Based Spatial Clustering of Applications with Noise – algorytm grupowania oparty na gęstości punktów w przestrzeni danych

DEMANDSENSING – demand sensing – podejście do prognozowania popytu wykorzystujące aktualne dane sprzedażowe i sygnały rynkowe do bieżącej aktualizacji prognoz

EOQ – Economic Order Quantity – model określający optymalną wielkość zamówienia minimalizującą koszty zamawiania i utrzymywania zapasów

ERP – Enterprise Resource Planning – system informatyczny wspierający zarządzanie zasobami przedsiębiorstwa

EiTI – Wydział Elektroniki i Technik Informatycznych

FILLRATE – Fill Rate – wskaźnik określający procent popytu zaspokojonego bez braków magazynowych

GO – Go – język programowania opracowany przez Google, przeznaczony do tworzenia wydajnych aplikacji systemowych i sieciowych

GOOGLE DRIVE – usługa przechowywania plików w chmurze umożliwiająca zapisywanie, synchronizację oraz udostępnianie danych

GRID – Grid Search – metoda strojenia hiperparametrów polegająca na przetestowaniu wszystkich kombinacji wartości z zadanej siatki

GITHUB – GitHub – internetowa platforma do przechowywania kodu źródłowego oraz zarządzania wersjami projektów programistycznych

HASKELL – Haskell – funkcyjny język programowania o silnym systemie typów

HCA – Hierarchical Cluster Analysis – hierarchiczna analiza skupień polegająca na stopniowym łączeniu lub dzieleniu klastrów

HETEROGENICZNE – heterogeniczne źródła danych – dane o różnej strukturze, formacie lub pochodzeniu

IOT – Internet of Things – Internet Rzeczy; sieć urządzeń fizycznych wyposażonych w czujniki i oprogramowanie umożliwiające zbieranie oraz wymianę danych

JAVA – Java – obiektowy język programowania używany do tworzenia aplikacji wieloplatformowych

JIT – Just-In-Time – metoda zarządzania produkcją i zapasami polegająca na dostarczaniu materiałów dokładnie w momencie ich potrzebowania

JSON – JavaScript Object Notation – tekstowy format zapisu danych wykorzystywany do wymiany informacji między systemami

JS – JavaScript – język programowania wykorzystywany głównie do tworzenia aplikacji internetowych

JULIA – Julia – język programowania przeznaczony do obliczeń naukowych i analizy danych

JUPYTER – Jupyter Notebook – interaktywne środowisko programistyczne umożliwiające uruchamianie kodu, wizualizację wyników oraz dokumentowanie analiz

KAGGLE – internetowa platforma udostępniająca zbiory danych oraz środowisko do analizy danych i uczenia maszynowego

KERNEL – kernel – funkcja jądra wykorzystywana w algorytmach takich jak SVM do odwzorowania danych w przestrzeni o wyższym wymiarze

KMEANS – metoda k-średnich – algorytm grupowania danych dzielący obserwacje na k klastrów na podstawie podobieństwa

KNN – K-Nearest Neighbors Regression – metoda regresji oparta na przewidywaniu wartości na podstawie średniej wartości dla k najbardziej podobnych obserwacji

KPCA – Kernel Principal Component Analysis – jądrowa analiza głównych składowych wykorzystująca funkcję jądra

Lasso – Lasso Regression – regresja liniowa z regularyzacją L1

LEARNINGRATE – learning_rate – parametr określający szybkość uczenia modelu, kontrolujący wielkość zmian parametrów podczas procesu uczenia

LIGHTGBM – Light Gradient Boosting Machine – algorytm uczenia maszynowego oparty na metodzie gradient boosting wykorzystujący drzewa decyzyjne

LISP – Lisp – rodzina języków programowania wykorzystywanych m.in. w badaniach nad sztuczną inteligencją

LR – Linear Regression – model regresji liniowej

LSTM – Long Short-Term Memory – typ rekurencyjnej sieci neuronowej wykorzystywanej do analizy danych sekwencyjnych, np. szeregów czasowych

MA – Moving Average – metoda średniej ruchomej polegająca na prognozowaniu na podstawie średniej z ostatnich obserwacji

MAE – Mean Absolute Error – średni błąd bezwzględny prognozy

MAPE – Mean Absolute Percentage Error – średni bezwzględny błąd procentowy

ML – Machine Learning - uczenie maszynowe

MLP – Multilayer Perceptron – wielowarstwowa sieć neuronowa typu feedforward

MULTIMODALNE – multimodalne źródła danych – dane pochodzące z różnych typów źródeł, np. tekstu, obrazów, danych liczbowych lub sygnałów

NAIVE – Naive forecasting – metoda prognozowania zakładająca, że wartość prognozy jest równa ostatniej obserwacji

NB – Naive Bayes – probabilistyczny algorytm uczenia maszynowego oparty na twierdzeniu Bayesa i założeniu niezależności cech

NEURON – podstawowa jednostka obliczeniowa w sztucznej sieci neuronowej, przetwarzająca sygnały wejściowe za pomocą wag i funkcji aktywacji

NESTIMATORS – n_estimators – parametr modeli zespołowych określający liczbę drzew decyzyjnych w modelu

NUMPY – NumPy – biblioteka języka Python wykorzystywana do obliczeń numerycznych i pracy z tablicami wielowymiarowymi

OCSVM – One-Class Support Vector Machine – jednoklasowa maszyna wektorów nośnych stosowana do wykrywania obserwacji odstających

OTIF – On Time In Full – wskaźnik określający procent zamówień dostarczonych na czas i w pełnej ilości

PANDAS – Pandas – biblioteka języka Python służąca do analizy i przetwarzania danych tabelarycznych

PCA – Principal Component Analysis – analiza głównych składowych, metoda redukcji wymiarowości danych

PDQ – parametry modelu ARIMA: p – rząd części autoregresyjnej, d – stopień różnicowania szeregu czasowego, q – rząd części średniej ruchomej

PKL – plik w formacie Pickle, wykorzystywany w języku Python do zapisu i odczytu obiektów pośrednich

POS – Point of Sale – punkt sprzedaży; dane sprzedażowe rejestrowane bezpośrednio w miejscu dokonania transakcji

PYPL – Popularity of Programming Language Index – indeks popularności języków programowania oparty na liczbie wyszukiwań w Google

PYTHON – język programowania wysokiego poziomu wykorzystywany m.in. w analizie danych i uczeniu maszynowym

R – R – język programowania oraz środowisko obliczeniowe stosowane w analizie statystycznej i wizualizacji danych

R² – współczynnik determinacji; miara jakości modelu określająca, jaka część zmienności danych jest wyjaśniana przez model; obliczany jako $R^2 = 1 - \frac{\text{suma kwadratów błędów modelu}}{\text{suma kwadratów odchyleń od średniej}}$

RANDOM – Random Search – metoda strojenia hiperparametrów polegająca na losowym wyborze kombinacji parametrów

RBF – Radial Basis Function – funkcja bazowa radialna stosowana m.in. w metodach jądrowych

RF – Random Forest – model zespołowy oparty na wielu drzewach decyzyjnych

Ridge – Ridge Regression – regresja liniowa z regularyzacją L2

RMSE – Root Mean Squared Error – pierwiastek średniego błędu kwadratowego

RNN – Recurrent Neural Network – rekurencyjna sieć neuronowa stosowana do przetwarzania danych sekwencyjnych

RT – Regression Tree – model regresyjny w postaci drzewa decyzyjnego dzielącego dane na kolejne podzbiory

SCALA – Scala – język programowania działający na platformie JVM, łączący paradygmat obiektowy i funkcyjny

SCM – Supply Chain Management – zarządzanie przepływem materiałów, informacji i produktów w łańcuchu dostaw

SES – Simple Exponential Smoothing – metoda wygładzania wykładniczego wykorzystująca malejące wagi dla starszych obserwacji

SKLEARN – Scikit-learn – biblioteka języka Python zawierająca implementacje algorytmów uczenia maszynowego

SVM – Support Vector Machine – metoda uczenia maszynowego oparta na maksymalizacji marginesu

SVR – Support Vector Regression – wersja regresyjna algorytmu Support Vector Machine

TF – TensorFlow – biblioteka programistyczna do budowy i trenowania modeli uczenia maszynowego

TIOBE – TIOBE Index – ranking popularności języków programowania oparty m.in. na liczbie wyników wyszukiwania w Internecie

WMS – Warehouse Management System – system zarządzania operacjami magazynowymi

XAI – Explainable Artificial Intelligence – podejście do sztucznej inteligencji umożliwiające interpretację i wyjaśnianie decyzji modeli uczenia maszynowego

XGBOOST – Extreme Gradient Boosting – algorytm uczenia maszynowego oparty na metodzie gradient boosting

Spis rysunków

3.1	Przykładowy szereg czasowy przedstawiający miesięczną sprzedaż produktu .	11
4.1	Rodzaje uczenia maszynowego [13]	15
6.1	Porównanie wartości RMSE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym	33
6.2	Porównanie wartości MAE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym	34
6.3	Porównanie wartości MAPE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym	34
6.4	Porównanie wartości R^2 dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym	35
6.5	Porównanie wartości R^2 dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym	36
6.6	Porównanie wartości MAPE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym	36
6.7	Porównanie wartości MAE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym	37
6.8	Porównanie wartości RMSE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym	37
6.9	Porównanie wartości RMSE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym	38
6.10	Porównanie wartości MAE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym	39
6.11	Porównanie wartości MAPE dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym	39
6.12	Porównanie wartości R^2 dla modeli tradycyjnych oraz modeli uczenia maszynowego w wariancie statycznym i kroczącym	40
6.13	Porównanie wyników kontroli zapasów dla różnych modeli prognozowania – liczba dni braku zapasu oraz średni poziom zapasu	42
6.14	Porównanie wyników kontroli zapasów dla różnych modeli prognozowania – liczba dni braku zapasu oraz średni poziom zapasu (Men's Street Footwear) . .	43

Spis tabel

3.1	Rodzaje modeli szeregów czasowych[4]	11
4.1	Dane uczące	20
6.1	Próbka danych – Adidas US Sales Dataset	31
6.2	Próbka danych – Superstore Sales Dataset	32
Zał. 1.1	Test 1 – modele tradycyjne (statyczne)	57
Zał. 1.2	Test 1 – modele tradycyjne (kroczące / rolling)	57
Zał. 1.3	Test 1 – modele ML (statyczne)	58
Zał. 1.4	Test 1 – modele ML (kroczące / rolling)	58
Zał. 2.1	Test 2 – Women’s Street Footwear. Zestawienie wyników prognoz (modele statyczne i kroczące)	59
Zał. 3.1	Test 3 – dane „Superstore Sales Dataset 2016–18”	60
Zał. 4.1	Test kontroli zapasów – zbiór danych: Superstore Sales Dataset 2016–2018	61
Zał. 4.2	Test kontroli zapasów – Men’s Street Footwear (dane filtrowane)	61

Spis załączników

1.	Wyniki eksperymentów test 1	57
2.	Wyniki eksperymentów test 2	59
3.	Wyniki eksperymentów test 3	60
4.	Wyniki eksperymentów test 4 (kontroli zapasów)	61

Załącznik 1. Wyniki eksperymentów test 1

Tabela Zał. 1.1. Test 1 – modele tradycyjne (statyczne)

model	parametry	mae	rmse	mape	r2
naive	classic	4210,33	5697,25	93,03	-0,78
ma	window=3	4292,85	5797,89	89,74	-0,84
ma	window=7	4331,98	5844,38	88,34	-0,87
ma	window=21	4812,22	6380,5	77,13	-1,23
ses	$\alpha = 0.0$	3952,89	5307,66	111,01	-0,55
ses	$\alpha = 0.1$	4634,56	6190,63	79,86	-1,1
ses	$\alpha = 0.3$	4340,42	5854,25	88,06	-0,88
arima	(0,1,0)	4210,33	5697,25	93,03	-0,78
arima	(1,1,0)	4263,34	5762,05	90,88	-0,82
arima	(2,1,1)	4332,08	5844,46	88,34	-0,87

Tabela Zał. 1.2. Test 1 – modele tradycyjne (kroczące / rolling)

model	parametry	mae	rmse	mape	r2
naive	kroczący	4016	5397	185	-0,59
ma	window=3	3187	4558	178	-0,14
ma	window=7	3287	4197	192	0,03
ma	window=21	3759,03	4672,17	243,59	-0,2
ses	$\alpha = 0.1$	3444,83	4359	218,9	-0,04
ses	$\alpha = 0.3$	3248,79	4358,44	189,88	-0,04
arima	(1,1,1)	3245,5	4358,11	190,03	-0,04
arima	(2,1,1)	3233,09	4342,86	188,74	-0,03

Tabela Zał. 1.3. Test 1 – modele ML (statyczne)

model	tryb	parametry	mae	rmse	mape	r2
linreg	manual	3848	5459	87	-0,63	
ridge	manual	$\alpha = 10$	1511,65	2042,07	64,18	0,77
lasso	manual	$\alpha = 100$	1695,84	2398,74	63,97	0,68
rf	random	n=100–800	2630,31	3324,18	130,91	0,39
svr	random	linear	2743,82	3864,81	73,36	0,18
xgboost	grid	–	2314,22	3010,45	94,19	0,5
xgboost	random	–	2157,93	2962,26	81,63	0,52
mlp	grid	–	3122,65	4316,47	82,29	-0,02

Tabela Zał. 1.4. Test 1 – modele ML (kroczące / rolling)

model	tryb	parametry	mae	rmse	mape	r2
linreg	manual	–	1240,26	1741,73	35,12	0,83
ridge	random	$\alpha \in [0.01, 100]$	1211,38	1654,25	34,11	0,85
lasso	manual	$\alpha = 10$	1219,35	1690,67	35,07	0,84
rf	manual	n=300, depth=10	1191,64	1707,9	32,67	0,84
svr	manual	linear, C=100	1238,17	1808,81	30,93	0,82
xgboost	manual	n=200, depth=4	1179,65	1673,32	37,34	0,85
mlp	grid	–	2090,52	2753,13	98,15	0,58

Załącznik 2. Wyniki eksperymentów test 2

Tabela Zał. 2.1. Test 2 – Women's Street Footwear. Zestawienie wyników prognoz (modele statyczne i kroczące)

model	typ prognozy	parametry	mae	rmse	mape	r2
linear regression	statyczna	manual	468.72	616.61	3504.97	0.374
naive	statyczna	klasyczna	629.71	826.84	13100.47	-0.125
moving average	krocząca	okno = 7	635.32	789.28	16168.63	-0.025
ses	krocząca	$\alpha = 0.3$	657.20	829.31	16070.11	-0.132
arima	krocząca	(2,1,1)	652.67	837.04	17374.33	-0.153
ridge regression	statyczna	$\alpha = 10$	445.88	565.25	4552.94	0.474
lasso regression	statyczna	$\alpha = 100$	317.62	453.82	2273.56	0.661
random forest	statyczna	random search	582.48	803.49	80.74	-0.062
xgboost	statyczna	random search	544.35	730.36	860.49	0.122
xgboost	statyczna	grid search	491.92	646.74	2384.42	0.312
svr	statyczna	linear, C = 100	332.70	416.44	6897.63	0.715
linear regression	krocząca	–	266.23	351.45	5015.53	0.797
ridge regression	krocząca	$\alpha = 10$ / random	251.66	332.76	2939.02	0.818
lasso regression	krocząca	$\alpha = 10$	275.80	366.23	1696.25	0.779
svr	krocząca	linear, C = 100	292.08	373.99	4870.98	0.770
random forest	krocząca	500 drzew, depth = 20	251.24	361.39	54.88	0.785
xgboost	krocząca	n = 200, depth = 4	247.13	341.46	1444.92	0.808
mlp	statyczna	(100), relu, $\alpha = 0.001$	566.59	770.37	9896.90	0.024

Załącznik 3. Wyniki eksperymentów test 3

Tabela Zał. 3.1. Test 3 – dane „Superstore Sales Dataset 2016–18”

model	tryb	parametry	mae	rmse	mape	r2
naive	statyczne	–	740,01	1863,6	34,72	-0,187
linear regression	statyczne	manual	850,07	1499,37	35480,51	0,232
moving average	statyczne	window = 7	961,26	1774,39	39943,8	-0,076
ses	statyczne	$\alpha = 0.3$	909,61	1707,18	35446,14	0,004
arima	statyczne	(2,1,1)	910,27	1649,64	33731,38	0,07
ridge	statyczne	$\alpha = 10$	837,49	1495,72	34380,43	0,235
lasso	statyczne	$\alpha = 100$	785,9	1501,09	28347,48	0,23
random forest	statyczne	grid search	571,81	1255,66	9954,08	0,461
xgboost	statyczne	grid search	587,83	1359,65	10375,69	0,368
svr	statyczne	linear, C = 100	1650,74	2275,36	100752,04	-0,77
svr	statyczne	linear, C = 10	745,07	1793,3	6019,73	-0,099
mlp	statyczne	grid search	757,35	1594,78	19503,09	0,131
linear regression	kroczące	–	871	1512,11	37900,71	0,218
ridge	kroczące	$\alpha = 10$	861,07	1505,23	37442,91	0,225
lasso	kroczące	$\alpha = 100$	810,87	1498,5	32149,75	0,232
svr	kroczące	linear, C = 100	2141,61	2772,72	151682,93	-1,628
xgboost	kroczące	n = 200, depth = 4	627,15	1454,91	12200,94	0,276
mlp	kroczące	random search	806,39	1530,28	28779,1	0,2
svr	kroczące	random search	995,95	1739,95	39771,06	-0,035
xgboost	kroczące	grid search	573,44	1321,22	11713,39	0,403
random forest	kroczące	grid search	556,3	1361,19	8980,73	0,367

Załącznik 4. Wyniki eksperymentów test 4 (kontroli zapasów)

Tabela Zał. 4.1. Test kontroli zapasów – zbiór danych: Superstore Sales Dataset 2016–2018

model	tryb prognozy	parametry	dni braku zapasu	średni stan zapasu
random forest	statyczne	grid search	44	5529
xgboost	kroczące	grid search	50	4717
xgboost	statyczne	grid search	54	4296
random forest	kroczące	grid search	49	5065
arima	kroczące	(2,1,1)	82	2980
ses	kroczące	$\alpha = 0.3$	50	5877

Tabela Zał. 4.2. Test kontroli zapasów – Men's Street Footwear (dane filtrowane)

model	tryb prognozy	parametry	stockout_days	avg_stock
ridge regression	kroczący	$\alpha = 10$ / random search	57	4315
xgboost	kroczący	grid search	66	3297
linear regression	kroczący	–	56	4605
random forest	kroczący	grid search	59	4249
svr	statyczny	kernel = linear, random	90	2882
lasso	statyczny	grid search	53	4350
ridge regression	statyczny	grid search	56	4340
moving average	kroczący	window = 7	61	4291
ses	kroczący	$\alpha = 0.3$	59	4399